fractal and fractional

MDPI

Article

Fractional-Order Memory and Elite-Center-Guided Sine–Cosine Optimization for Feature Selection

Yuhang Xie [1], Wei Li [1], Bin Qin [1], Kai Xu [1] and Shang Gao [2,*]

[1] School of Computer, Jiangsu University of Science and Technology, Zhenjiang 212003, China; 232211901130@stu.just.edu.cn (Y.X.)
[2] Ocean College, Jiangsu University of Science and Technology, Zhenjiang 212003, China
* Correspondence: gao_shang@just.edu.cn

Abstract

Metaheuristic optimization algorithms are widely used in wrapper-based feature selection, since they can search complex combinatorial spaces without gradient information. The standard sine cosine algorithm (SCA) is simple and easy to implement, but depends on the current population state and a single global optimal guide. This dependence can cause premature convergence and late-stage oscillations in discretized feature-selection tasks. To address these issues, we propose the fractional-order elite-memory sine cosine algorithm (FOSCA). The FOSCA integrates short-memory fractional-order position reconstruction, dynamic elite-center guidance and nonlinear search-factor decay into the SCA search dynamics. These mechanisms improve the trajectory continuity, guidance diversity and convergence stability. Experiments on 14 classification datasets showed that the FOSCA achieved a competitive accuracy, F1-score, precision and recall. The FOSCA also outperformed the original SCA on 12 out of the 14 datasets. Statistical tests, a stability analysis, a performance–sparsity trade-off analysis and ablation studies confirmed the effectiveness of the proposed mechanisms. Overall, the FOSCA improves the SCA search reliability in discrete feature selection and offers a reproducible basis for related optimizer-based methods.

Keywords: feature selection; wrapper-based; SCA; fractional order; elite memory; metaheuristic optimization; dynamic elite guidance

1. Introduction

Feature selection is a central problem in machine learning and data mining, since modern datasets often contain redundant, irrelevant, or noisy features that degrade the classification performance [1–3]. By selecting a compact subset of informative features, feature selection reduces the computational cost, improves the model interpretability and helps alleviate overfitting [1,4,5]. Wrapper-based feature selection evaluates each candidate subset through the predictive performance of a learning algorithm [6,7]. This design aligns the selected features with the target classification task. However, for a dataset with (D) original features, the search space contains (2^D) possible subsets [6]. Feature selection is therefore a challenging discrete combinatorial optimization problem. Designing an effective search strategy remains valuable for classification modeling and intelligent optimization, especially under a limited evaluation budget [4,7,8].

Prior work has advanced feature selection from several perspectives. Classical methods are generally grouped into filter, wrapper and embedded methods [9]. Filter methods

Academic Editor: Inés Tejado

Received: 29 June 2026
Revised: 25 July 2026
Accepted: 29 July 2026
Published: 31 July 2026

select features independently of a specific classifier, using statistical or information-theoretic criteria [2,10]. Common metrics include the correlation coefficient, chi-square test, mutual information, variance threshold and information gain, which measure the importance of individual features or feature groups [11,12]. These methods are computationally efficient and suitable for large-scale data, but they often ignore feature interactions and do not directly optimize the final classifier performance [5]. Embedded methods integrate feature selection into model training, as in the LASSO and tree-based learning models [9]. They often balance the efficiency and predictive performance, but their effectiveness depends closely on the learning model structure [13]. In contrast, wrapper methods evaluate candidate feature subsets with a learning algorithm and search for subsets that improve the predictive performance [14,15]. They are usually more computationally expensive, but often provide stronger task adaptability.

Metaheuristic and swarm intelligence algorithms are widely used in wrapper-based feature selection. They require no gradient information, handle discrete search spaces flexibly and balance the classification accuracy with the subset sparsity [16–18]. Representative optimizers include genetic algorithms, particle swarm optimization [19], differential evolution [20], the gray wolf optimizer [21], the whale optimization algorithm [22], the Harris hawks optimizer [23] and the sine cosine algorithm [24–26]. Across classification tasks, such optimizers have produced a promising feature-subset search performance [27]. For example, binary gray wolf optimization adapts to a feature subset search [28]. Wrapper-based WOA variants reduce the number of selected features while maintaining the classification accuracy [29]. Binary SCA variants use S-shaped and V-shaped transfer functions to convert continuous positions into binary feature vectors for medical feature selection [30]. Fractional calculus has also been incorporated into population-based optimizers, including the fractional-order velocity in PSO [31] and the Riesz fractional mutation in the SCA [32]. Fractional memory, elite guidance, and nonlinear control are established design elements rather than independently novel optimization principles. Related studies have advanced the field through flexible search frameworks, binary transformations, and hybrid wrapper models [30,33].

Although metaheuristic feature-selection methods have made considerable progress, several limitations remain [34]. The search process still faces premature convergence, insufficient exploration in high-dimensional spaces, unstable binary mapping and difficulty with balancing the classification performance with the subset sparsity [35]. The mapping from continuous optimizer positions to binary feature subsets can also amplify random perturbations. This may cause unstable feature switching, redundant subset expansion or late-stage oscillations [36]. These limitations motivate optimizer-specific mechanisms that exploit useful historical information, maintain the population diversity and stabilize late-stage searches.

These issues are particularly relevant to the SCA [37]. The SCA has a simple structure and uses sine–cosine perturbations to guide candidate solutions toward or away from the optimal solution. However, its update mechanism mainly depends on the current population state and a single global optimal guide. The original linearly decreasing search factor may also be insufficient for both early exploration and stable late-stage exploitation in discrete feature selection. It is therefore necessary to redesign the SCA search dynamics for feature-selection through stronger memory utilization and search-amplitude control [38].

To address these issues, we propose the FOSCA, a fractional-order elite-memory sine cosine algorithm for wrapper-based feature selection. The FOSCA keeps the external wrapper evaluation protocol unchanged and improves the internal SCA search dynamics before binary feature subset generation. Specifically, it introduces short-memory fractional-order position reconstruction to build the update basis from the current state and recent historical

states. This mechanism improves the continuity of the search trajectories. The FOSCA also constructs a fitness-weighted elite center whose membership contracts over time and whose position progressively shifts toward the global-optimal solution. As the search proceeds, the guide gradually shifts toward the global optimal solution [39]. In addition, an alpha-coupled nonlinear search-factor decay strategy preserves sufficient early-stage perturbations and reduces unnecessary late-stage oscillations. The RFSCA [32] applies a fractional mutation to the best individual, whereas the MG-SCA [40] and EPSCA [41] use stored or pooled guides. In contrast, the FOSCA jointly modifies each candidate's update basis, guidance target, and perturbation amplitude before binary mapping and greedy acceptance. Through these mechanisms, the FOSCA preserves the simplicity of the SCA while improving the stability, diversity and convergence reliability for wrapper-based feature selection.

The main contributions of this paper are summarized as follows:

1. We propose the FOSCA, a fractional-order elite-memory SCA framework for wrapper-based feature selection. The FOSCA embeds a short-memory fractional-order mechanism into the continuous SCA search dynamics. This allows position updates to use recent historical search information before binary subset generation.
2. We designed a dynamic elite-center guidance mechanism to reduce the SCA's dependence on a single global optimal solution. The mechanism constructs a weighted guide from multiple high-quality individuals and gradually contracts it toward the global optimal solution. This improves the balance between population-level exploration and local exploitation.
3. We developed an alpha-coupled nonlinear search-factor decay strategy to control the perturbation amplitude of the SCA. The strategy maintains a broad search range in the early stage and reduces late-stage oscillations. This makes the search process more suitable for discretized feature selection.
4. We conducted extensive experiments under a unified function-evaluation budget to compare the FOSCA with representative metaheuristic algorithms and SCA variants. We evaluated the proposed mechanisms using multiple classification performance metrics, the number of selected features, the statistical significance and an ablation analysis.

The remainder of this paper is organized as follows: Section 2 presents the overview of the conventional sine cosine algorithm and the motivation underlying the proposed algorithm. Section 3 introduces the overall framework and core mechanisms of the FOSCA. Section 4 presents the experimental setup and the associated result analysis. Section 5 concludes the paper and outlines possible directions.

2. Related Work

Section 2 reviews the SCA, summarizes representative variants, and identifies limitations relevant to wrapper-based feature selection. The review establishes the need for developing a more stable and reliable SCA-based feature-selection method.

2.1. Sine Cosine Algorithm and Its Variants

The sine cosine algorithm (SCA) is a population-based metaheuristic optimizer for solving optimization problems [37]. In the standard SCA, each candidate updates its position based on its distance from the current optimal solution. Sine and cosine functions control the movement direction and step size. A linearly decreasing control parameter shifts the search from exploration to exploitation. Owing to its simple structure, few parameters and ease of implementation, the SCA has been applied to numerical optimization, engineering design [42], image processing and feature selection [43].

Several SCA variants address the limitations of the standard algorithm [44,45]. Representative SCA variants combine opposition learning, a neighborhood search, crossover operators and multi-strategy updates to improve the convergence and reduce local trapping [38,46]. The literature can be organized into five lines of development, each targeting a different part of the search process. Adaptive and nonlinear-control variants modify the population roles or stage-dependent parameters [47,48]. Opposition-based [47] and chaotic variants broaden search sampling [49]. Population-structured methods construct elite pools [41] or heterogeneous subpopulations [48]. Other variants introduce Lévy flights [50] or hybridize the SCA with adaptive differential-evolution operators [51]. Fractional and memory-guided variants redesign elite mutations or progressively reduce the number of search guides [32,40].

Abd Elaziz et al. introduced opposition-based learning into the SCA to increase the population diversity and strengthen global searches [26]. Gupta et al. developed the mSCA with a nonlinear transition parameter, an elite-guided search and a mutation operator to improve the exploration–exploitation balance [25]. Nadimi-Shahraki et al. proposed the MTV-SCA, which uses multiple trial-vector generation strategies to reduce local-optima stagnation and improve the performance on complex optimization problems [24]. More recent variants include the ASCA, EPSCA, ICSCA, JADESCA and TS-SESCA [41,47–49,51].

For feature selection, the SCA usually requires binary mapping, as the original algorithm searches a continuous space [30]. Recent SCA-based methods combine triangular optimization with a feature-subset search [52]. Other binary optimizers employ adaptive step-size control or performance-driven role reassignment [53,54]. Together, these studies show that the SCA offers a flexible framework whose performance depends on its update mechanism, guidance strategy and binary conversion process.

Despite these advances, the standard SCA remains limited for wrapper-based feature selection. First, a single optimal solution mainly guides the population. Guidance by a single incumbent best can cause premature aggregation around a locally promising region [37,44]. Second, the original position update ignores historical search trajectories. The search may therefore become sensitive to random perturbations and unstable after binarization [46]. Third, the linearly decreasing search factor offers only a simple transition from exploration to exploitation. It may not sufficiently suppress late-stage oscillations in a discrete feature-subset search. Consequently, some feature bits may be repeatedly added or removed after the population reaches promising regions. Collectively, single-guide dependence, an absent memory, and linear decay motivate a more stable SCA-based feature-selection method. The FOSCA differs from existing variants by coordinating short-term temporal reconstruction, evaluation-dependent elite contraction and α-coupled amplitude decay before fixed-threshold binarization.

2.2. Motivation for FOSCA

The FOSCA was motivated by the need for both strong global exploration and stable subset refinement in feature selection. In discrete feature selection, excessive randomness can cause unstable feature switching. Excessive exploitation can also trap the population in a locally optimal subset. Therefore, an effective SCA-based feature-selection method should reduce the dependence on a single optimal guide, exploit useful historical search information and adjust the perturbation amplitude according to the search stage [55]. The FOSCA addresses these requirements by integrating short-memory fractional-order position reconstruction, a dynamic elite-center guide and alpha-coupled nonlinear search-factor decay into the continuous SCA search dynamics. Their combined operation improves the trajectory continuity and early directional diversity while suppressing late oscillations and rejecting degraded candidates. Thus, the FOSCA systematically enhances the SCA for

wrapper-based feature selection rather than merely adjusting parameters or modifying the external evaluation.

3. The Proposed FOSCA Methodology

This section presents the fractional-order elite-memory sine cosine algorithm (FOSCA). The FOSCA introduces three complementary mechanisms: short-memory fractional position reconstruction, dynamic elite-center guidance and an alpha-coupled fast-decaying exploration factor. The FOSCA integrates these strategies into the SCA search dynamics. It preserves the structural simplicity of the SCA and the flexibility of sine–cosine perturbations while improving exploration, exploitation and convergence stability in the feature-selection space.

3.1. Feature Subset Representation and Objective Function

Let

$$\mathcal{D} = \{(\mathbf{a}_m, y_m)\}_{m=1}^{M}, \tag{1}$$

denote a classification dataset with M samples. Here, $a_m \in \mathbb{R}^D$ is the m-th feature vector, and y_m is its class label. Each candidate solution is encoded as a continuous position vector in:

$$\mathbf{x}_i^t = (x_{i,1}^t, x_{i,2}^t, \ldots, x_{i,D}^t) \in [0,1]^D, \tag{2}$$

where $i = 1, 2, \ldots, N$ indicates the index of the i-th individual in the population, N is the population size, t denotes the current update iteration and D is the number of original features. Since feature selection is essentially a binary combinatorial optimization problem, the continuous position vector should be further transformed into a binary feature-selection mask. In this study, a fixed thresholding strategy was adopted to perform this mapping:

$$b_{i,j}^t = \begin{cases} 1, & x_{i,j}^t > 0.5, \\ 0, & x_{i,j}^t \le 0.5, \end{cases} \quad j = 1, 2, \ldots, D, \tag{3}$$

where $b_{i,j}^t = 1$ indicates that the j-th feature is selected, whereas $b_{i,j}^t = 0$ means that the feature is discarded. Accordingly, the feature subset represented by the i-th candidate solution can be defined as

$$S_i^t = \{j \mid b_{i,j}^t = 1,\ j = 1, 2, \ldots, D\}. \tag{4}$$

To avoid the infeasible case in which an empty feature subset prevents the classifier from being trained, an empty-subset repair rule was applied. Specifically, if no feature is selected by a candidate solution, the feature with the largest continuous position value is forced to be selected:

$$j^* = \arg\max_{1 \le j \le D} x_{i,j}^t, \quad b_{i,j^*}^t = 1. \tag{5}$$

The objective of feature selection is to reduce redundant features while preserving the classification performance. Therefore, the fitness function was constructed by combining the classification error rate and the selected feature ratio:

$$f(\mathbf{x}_i^t) = (1 - \lambda) MCE_{\mathrm{CV}}(S_i^t) + \lambda \frac{|S_i^t|}{D}, \tag{6}$$

where $f(\mathbf{x}_i^t)$ denotes the fitness value of the i-th candidate solution, $\lambda = 0.01$ is the penalty coefficient for the number of selected features, $|S_i^t|$ is the number of selected features and D represents the total number of original features. $MCE_{\mathrm{CV}}(S_i^t)$ denotes the average misclassification error obtained by K-fold cross-validation on the training set, which is defined as Equation (7):

$$MCE_{\mathrm{CV}}(S) = \frac{1}{K}\sum_{k=1}^{K}\frac{1}{|\mathcal{V}_k|}\sum_{(\mathbf{a}_m, y_m)\in\mathcal{V}_k}\mathbb{I}(h_k(\mathbf{a}_{m,S}) \neq y_m), \tag{7}$$

where K is the number of cross-validation folds, $\mathcal{V}_k$ denotes the validation subset in the k-th fold and $|\mathcal{V}_k|$ is the number of samples in that validation subset. $\mathbf{a}_{m,S}$ represents the projection of sample $\mathbf{a}_m$ onto the selected feature subset S, $h_k(\cdot)$ denotes the classifier trained on the corresponding training folds in the k-th cross-validation process and $\mathbb{I}(\cdot)$ is the indicator function. The indicator function returns 1 when the predicted label is different from the true label and returns 0 otherwise. According to Equation (6), a smaller fitness value corresponds to a feature subset with a lower classification error and a smaller number of selected features. Therefore, the feature-selection task was formulated as a minimization optimization problem in this study.

3.2. Basic SCA Update Mechanism

The original sine cosine algorithm (SCA) updates individual positions by using sine and cosine functions to drive the population around the current optimal solution. Let $\mathbf{g}^t$ denote the individual with the best fitness value in the current population. The position update of the original SCA in the j-th dimension can be formulated as shown in Equation (8):

$$x_{i,j}^{t+1} = \begin{cases} x_{i,j}^t + r_1^t \sin(r_{2,i,j}^t)\left|r_{3,i,j}^t g_j^t - x_{i,j}^t\right|, & r_{4,i,j}^t < 0.5, \\ x_{i,j}^t + r_1^t \cos(r_{2,i,j}^t)\left|r_{3,i,j}^t g_j^t - x_{i,j}^t\right|, & r_{4,i,j}^t \geq 0.5. \end{cases} \tag{8}$$

where r_1^t is the factor controlling the search amplitude, $r_{2,i,j}^t$ determines the search direction generated by the sine or cosine function, $r_{3,i,j}^t$ adjusts the stochastic weight assigned to the target position and $r_{4,i,j}^t$ is used to switch between the sine-based and cosine-based update modes. Although this mechanism provides a strong stochastic search capability, it mainly relies on a single global optimal solution $\mathbf{g}^t$ as the guiding center. When the current optimal solution is located in a locally promising region, the population may prematurely concentrate around this region. In addition, the original SCA does not explicitly exploit historical search information, which may lead to unstable changes when candidate solutions are mapped from the continuous search space to binary feature subsets.

To alleviate these limitations, the FOSCA redesigns the update basis, guidance center and search-amplitude scheduling in Equation (8). Specifically, the FOSCA first reconstructs the current search state through a short-memory fractional mechanism. It then replaces the single optimal guide in the early search stage with a dynamic elite center and further adopts a fractional-order-coupled fast-decaying factor to reduce ineffective oscillations in the later search stage.

3.3. Short-Memory Fractional Position Reconstruction

The standard SCA updates each individual using only its current position. The FOSCA instead constructs a short-memory position from the backward Grünwald–Letnikov (GL) fractional difference [47]. For a unit iteration interval, the GL difference at iteration $t+1$ is

$$\Delta^{\alpha}\mathbf{x}_i^{t+1} = \sum_{k=0}^{t+1} \omega_k^{(\alpha)} \mathbf{x}_i^{t+1-k}, \qquad \omega_k^{(\alpha)} = (-1)^k \binom{\alpha}{k}, \tag{9}$$

where

$$\binom{\alpha}{k} = \frac{\alpha(\alpha-1)\cdots(\alpha-k+1)}{k!}, \qquad 0 < \alpha < 1. \tag{10}$$

Expanding the first four terms gives

$$\Delta^{\alpha}\mathbf{x}_i^{t+1} = \mathbf{x}_i^{t+1} - \alpha\mathbf{x}_i^{t} - \frac{\alpha(1-\alpha)}{2}\mathbf{x}_i^{t-1} - \frac{\alpha(1-\alpha)(2-\alpha)}{6}\mathbf{x}_i^{t-2} + \cdots. \tag{11}$$

Rearranging this expression separates the next state from its historical contribution:

$$\mathbf{x}_i^{t+1} = \Delta^{\alpha}\mathbf{x}_i^{t+1} - \sum_{k=1}^{t+1} \omega_k^{(\alpha)} \mathbf{x}_i^{t+1-k}. \tag{12}$$

The FOSCA extracts this historical contribution and applies a fixed short-memory truncation. Retaining the first three historical terms, $k = 1, 2, 3$, gives

$$\mathbf{x}_{\mathrm{mem},i}^{t} = -\sum_{k=1}^{3} \omega_k^{(\alpha)} \mathbf{x}_i^{t+1-k} = c_0\mathbf{x}_i^{t} + c_1\mathbf{x}_i^{t-1} + c_2\mathbf{x}_i^{t-2}, \tag{13}$$

where the historical coefficients satisfy $c_j = -\omega_{j+1}^{(\alpha)}$. Therefore,

$$c_0 = \alpha, \quad c_1 = \frac{1}{2}\alpha(1-\alpha), \quad c_2 = \frac{1}{6}\alpha(1-\alpha)(2-\alpha), \quad 0 < \alpha < 1. \tag{14}$$

In this study, α was set to 0.82, giving $(c_0, c_1, c_2) = (0.8200, 0.0738, 0.0290)$. As indicated by Equations (13) and (14), the fractional memory position is a weighted combination of the current search state and recent historical states. The current position remains dominant, while the two preceding states provide smaller historical corrections. As $\alpha \to 1$, $c_0 \to 1$ and $c_1, c_2 \to 0$, recovering the current-position basis of the original SCA. This short-memory reconstruction may moderate abrupt stochastic perturbations and reduce frequent feature-bit switching after discretization.

3.4. Dynamic Elite-Center Guidance Strategy

The original SCA uses a single global optimal solution as the search guide. Although this strategy is beneficial for exploitation in the later search stage, it may reduce the population diversity during the early stage. To address this issue, the FOSCA introduces a dynamic elite-center guidance strategy, in which a more stable search direction is constructed from multiple high-quality individuals. Let FE denote the current number

of function evaluations and $FE_{\max}$ denote the maximum number of function evaluations. The search progress is defined as

$$\tau = \frac{FE}{FE_{\max}}, \quad 0 \le \tau \le 1. \tag{15}$$

In the implementation, τ is determined by the FE count at the beginning of each population update and remains unchanged during the N evaluations within that update. According to the search progress, the number of individuals participating in the elite-center construction is dynamically determined using Equation (16):

$$L^t = \max(1, \text{round}[L_0 - (L_0 - 1)\tau]), \quad L_0 = \max\left(1, \text{round}\left(\frac{N}{2}\right)\right), \tag{16}$$

where N is the population size and L_0 denotes the initial number of elite individuals used for constructing the guidance center. As shown in Equation (16), approximately half of the population can contribute to the guidance center in the early search stage, which helps maintain the multi-directional search capability. As FE increases, L^t gradually decreases to 1, enabling the algorithm to shift from population-level exploration to refined exploitation around promising regions.

Let $\mathcal{E}^t = \{q_1^t, q_2^t, \ldots, q_{L^t}^t\}$ denote the index set of the top L^t elite individuals in the current population, which satisfies

$$f(\mathbf{x}_{q_1^t}^t) \le f(\mathbf{x}_{q_2^t}^t) \le \cdots \le f(\mathbf{x}_{q_{L^t}^t}^t). \tag{17}$$

The worst fitness value in the current population is defined as

$$f_{\text{worst}}^t = \max_{1 \le i \le N} f(\mathbf{x}_i^t), \tag{18}$$

Based on this value, the weight assigned to the k-th elite individual is calculated using Equation (19):

$$\rho_k^t = \frac{f_{\text{worst}}^t - f(\mathbf{x}_{q_k^t}^t)}{\sum_{\ell=1}^{L^t}\left[f_{\text{worst}}^t - f(\mathbf{x}_{q_\ell^t}^t)\right]}, \quad k = 1, 2, \ldots, L^t. \tag{19}$$

where ρ_k^t represents the contribution weight of the k-th elite individual in the dynamic center. Since the fitness function is minimized in this study, an elite individual with a smaller fitness value obtains a larger value of $f_{\text{worst}}^t - f(\mathbf{x}_{q_k^t}^t)$ and therefore contributes more to the elite center. If the denominator in Equation (19) becomes zero or the weights are numerically unavailable, uniform weights are assigned to all elite individuals.

Accordingly, the dynamic elite center is formulated as shown in Equation (20):

$$\mathbf{E}^t = \sum_{k=1}^{L^t} \rho_k^t \mathbf{x}_{q_k^t}^t, \tag{20}$$

where $\mathbf{E}^t$ denotes the elite center constructed from multiple high-quality individuals.

Equation (19) assigns larger weights to fitter elites and satisfies $\rho_k^t \ge 0$ and $\sum_{\ell=1}^{L_t}$. Thus, E^t in Equation (20) remains within the convex hull of the selected elites. Normalized fitness differences make the weighting invariant to positive affine transformations and avoid an additional selection-pressure parameter. Uniform weighting ignores quality differences, rank-based weighting discards their magnitudes, and softmax weighting requires a temperature parameter and is scale-sensitive. Therefore, Equation (19) offers a

concise trade-off between diverse guidance and fitness-based exploitation, though it is not universally optimal.

To gradually focus the search on the current global optimal solution in the later stage, the FOSCA further blends the elite center with the global optimal individual through a nonlinear mixing strategy:

$$\mathbf{G}^t = (1-\eta^t)\mathbf{E}^t + \eta^t \mathbf{g}^t, \quad \eta^t = \tau^{3/2}, \tag{21}$$

where $\mathbf{G}^t$ denotes the dynamic guidance vector of the FOSCA, $\mathbf{g}^t$ is the current global optimal individual and η^t is the nonlinear mixing factor. When τ is small, η^t is close to 0 and $\mathbf{G}^t$ is mainly determined by multiple elite individuals. As τ gradually approaches 1, η^t increases and $\mathbf{G}^t$ progressively collapses toward $\mathbf{g}^t$. The resulting guide offers diverse early search directions and progressively concentrates exploitation around high-quality regions.

3.5. Alpha-Coupled Fast-Decaying Exploration Factor

The search-amplitude factor r_1^t is an important parameter in the SCA for controlling the transition between exploration and exploitation. The original SCA usually adopts a linear decay strategy to adjust r_1^t. Although this strategy is simple and easy to implement, it may still maintain relatively strong oscillations in the later search stage when applied to feature-selection tasks. To reduce ineffective changes in candidate feature subsets during the late stage of optimization, the FOSCA designs r_1^t as a nonlinear fast-decaying factor coupled with the fractional order α:

$$r_1^t = 2(1-\tau)^{1/\alpha}. \tag{22}$$

where $\tau = FE/FE_{\max}$ denotes the current search progress and $\alpha \in (0,1)$ is the fractional order. Since $1/\alpha > 1$, the proposed schedule decays faster than the conventional linear form as the search progresses, especially in the middle and late stages.

Equation (22) belongs to the power-law family $r_p(\tau) = 2(1-\tau)^p$, with $p = 1/\alpha$. For $0 < \alpha \leq 1$, $p \geq 1$. The schedule preserves the original bounds,

$$r_1(0,\alpha) = 2, \qquad r_1(1,\alpha) = 0. \tag{23}$$

It also recovers the linear SCA schedule at $\alpha = 1$. Equation (22) is therefore a parameter-coupled extension of the original control factor. For $0 < \tau < 1$, its first two derivatives are

$$\frac{\partial r_1}{\partial \tau} = -\frac{2}{\alpha}(1-\tau)^{1/\alpha-1} < 0, \qquad \frac{\partial^2 r_1}{\partial \tau^2} = \frac{2}{\alpha}\left(\frac{1}{\alpha}-1\right)(1-\tau)^{1/\alpha-2} \geq 0. \tag{24}$$

The schedule is thus monotonically decreasing and convex. For $0 < \alpha < 1$, its ratio to the linear schedule is

$$\frac{r_1(\tau,\alpha)}{2(1-\tau)} = (1-\tau)^{1/\alpha-1}. \tag{25}$$

This ratio decreases from one to zero as τ increases. Equation (22) therefore imposes progressively stronger amplitude suppression relative to the linear SCA. A faster-than-linear decay primarily suppresses middle- and late-stage oscillations.

The reciprocal mapping $p = 1/\alpha$ links the decay curvature to the existing fractional order without introducing another control parameter. Varying α continuously also recovers the original linear schedule at $\alpha = 1$. Such optimality would require a predefined schedule family and an explicit search-performance criterion. Section 4.6 further examines the effect of α through analytical sensitivity and empirical results.

3.6. Fractional Elite-Memory Sine–Cosine Update

By integrating the short-memory fractional mechanism, the dynamic elite-center guide and the fast-decaying exploration factor, the candidate position update of the FOSCA is defined as

$$\tilde{x}_{i,j}^{t+1} = x_{\mathrm{mem},i,j}^{t} + r_1^t \psi_{i,j}^t \left| r_{3,i,j}^t G_j^t - x_{i,j}^t \right|, \tag{26}$$

where $\tilde{x}_{i,j}^{t+1}$ denotes the candidate position of the i-th individual in the j-th dimension, $x_{\mathrm{mem},i,j}^{t}$ is the j-th component of the fractional memory position and G_j^t is the j-th component of the dynamic guidance vector. $r_{3,i,j}^t \sim U(0,2)$ is a random scaling coefficient and $\psi_{i,j}^t$ denotes the sine–cosine perturbation term. Specifically, $\psi_{i,j}^t$ is defined as shown in Equation (27):

$$\psi_{i,j}^t = \begin{cases} \sin(r_{2,i,j}^t), & r_{4,i,j}^t < 0.5, \\ \cos(r_{2,i,j}^t), & r_{4,i,j}^t \geq 0.5, \end{cases} \tag{27}$$

where $r_{2,i,j}^t \sim U(0,2\pi)$ and $r_{4,i,j}^t \sim U(0,1)$. Equation (26) indicates that the FOSCA no longer uses the current position $\mathbf{x}_i^t$ as the direct update basis. Instead, the sine–cosine perturbation is performed on the fractional memory position $\mathbf{x}_{\mathrm{mem},i}^t$, which incorporates recent historical search information. Meanwhile, the single global optimal guide in the original SCA is replaced by the dynamic guidance vector $\mathbf{G}^t$. In this way, each individual can be influenced by multiple elite directions in the early search stage and gradually guided toward the global optimal solution in the later stage.

Since the generated candidate position may exceed the predefined search boundary, a truncation-based boundary-handling strategy was adopted:

$$\hat{x}_{i,j}^{t+1} = \min\left\{ \max\left\{ \tilde{x}_{i,j}^{t+1}, lb_j \right\}, ub_j \right\}, \tag{28}$$

where lb_j and ub_j denote the lower and upper bounds of the j-th dimension, respectively. In the feature-selection formulation used in this study, all dimensions are bounded by $lb_j = 0$ and $ub_j = 1$. After boundary handling, the continuous candidate vector $\hat{\mathbf{x}}_i^{t+1}$ is converted into a binary feature subset using Equation (3), and its fitness value is then evaluated according to Equation (6).

As illustrated in Figure 1, FOSCA integrates fractional memory, elite guidance, and nonlinear decay to coordinate historical-state utilization, search-direction guidance, and progressive control of the search amplitude throughout the optimization process.

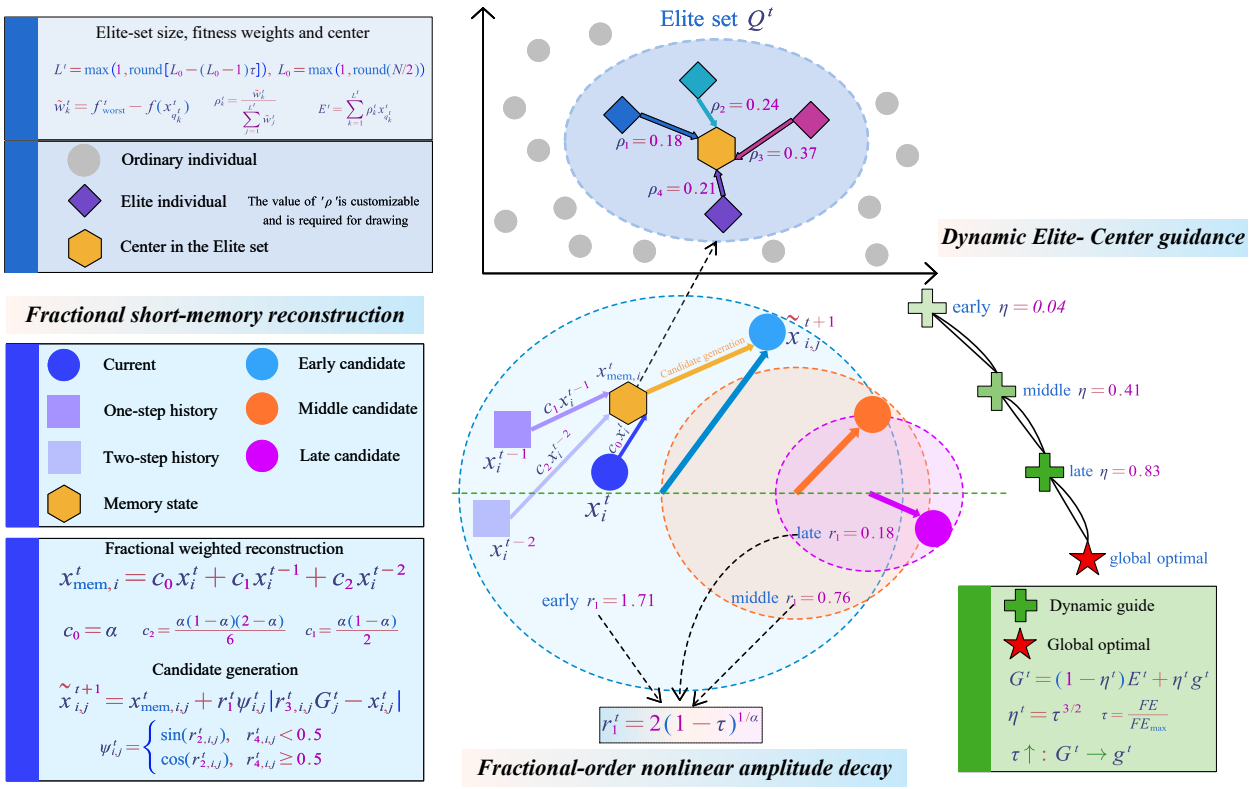

Figure 1. Mechanism diagram of the proposed FOSCA.

3.7. Greedy Individual-Level Acceptance

Since sine–cosine perturbations are inherently stochastic, unconditionally accepting all candidate positions may move individuals from promising regions to inferior regions, thereby weakening the convergence stability. To alleviate this problem, the FOSCA introduces a greedy individual-level acceptance mechanism after candidate generation and a fitness evaluation:

$$\mathbf{x}_i^{t+1} = \begin{cases} \hat{\mathbf{x}}_i^{t+1}, & f(\hat{\mathbf{x}}_i^{t+1}) \le f(\mathbf{x}_i^t), \\ \mathbf{x}_i^t, & f(\hat{\mathbf{x}}_i^{t+1}) > f(\mathbf{x}_i^t). \end{cases} \tag{29}$$

where $\hat{\mathbf{x}}_i^{t+1}$ denotes the candidate individual after boundary handling and $\mathbf{x}_i^{t+1}$ represents the finally accepted position for the next iteration. Greedy acceptance replaces an individual only with a candidate of equal or better fitness. By doing so, the FOSCA can reduce deteriorative moves caused by random perturbations and maintain a relatively stable local improvement tendency for each individual during the search process.

After all individuals complete candidate updating and greedy acceptance, the global optimal individual is updated as

$$\mathbf{g}^{t+1} = \arg\min_{\mathbf{z}\in\Omega} f(\mathbf{z}), \quad \Omega = \{\mathbf{g}^t, \mathbf{x}_1^{t+1}, \ldots, \mathbf{x}_N^{t+1}\}. \tag{30}$$

Meanwhile, the convergence curve is recorded with respect to the real number of function evaluations:

$$C(FE) = f(\mathbf{g}^t), \tag{31}$$

where $C(FE)$ denotes the best-so-far fitness value after the FE-th function evaluation. The indexing convergence by FE prevents nominal iterations with unequal evaluation counts from producing unfair algorithm comparisons.

The complete procedural flow of the FOSCA is illustrated in Figure 2. The pseudo-code of the proposed FOSCA is displayed in Algorithm 1.

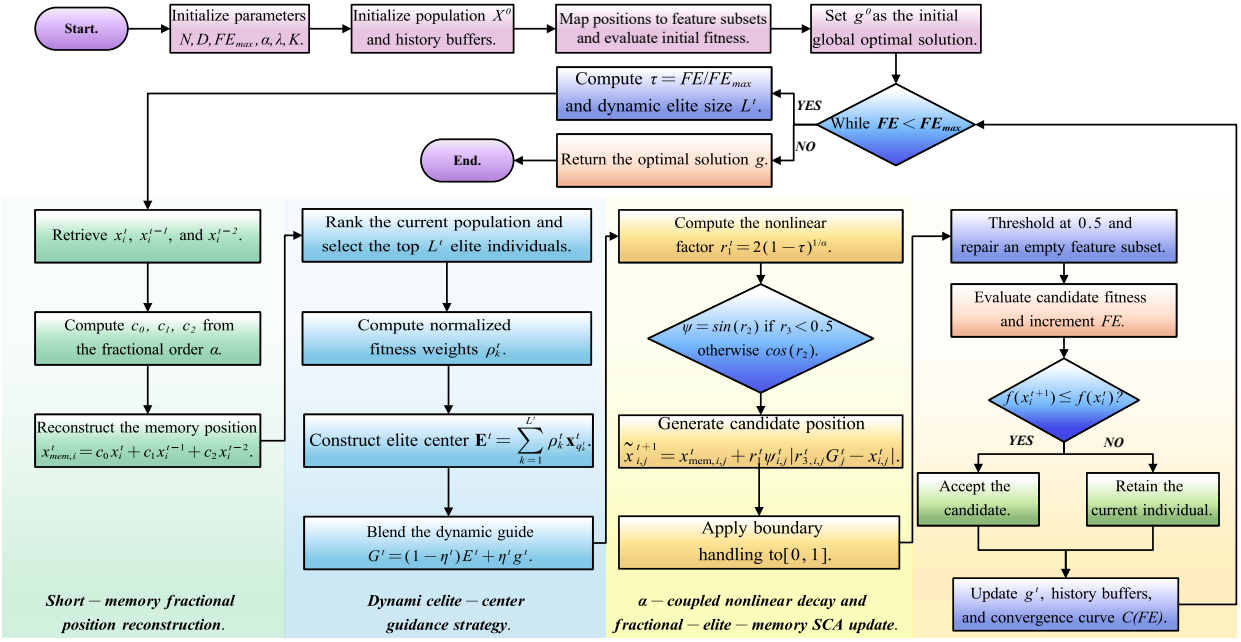

Figure 2. Flowchart of the proposed FOSCA.

Algorithm 1 Pseudo-code of FOSCA.

Require: Population size N, maximum function evaluations $FE_{\max}$, dimension D, bounds lb and ub, fitness function $f(\cdot)$, fractional order α.
Ensure: Best solution $\mathbf{g}$, best fitness $f(\mathbf{g})$, convergence curve C.
Initialize N individuals $\mathbf{x}_i^0 \in [lb, ub]^D$, $i = 1, 2, \ldots, N$.
Set the historical caches $\mathbf{x}_i^{-1} = \mathbf{x}_i^0$ and $\mathbf{x}_i^{-2} = \mathbf{x}_i^0$.
Evaluate each individual using the fitness function in Equation (6) and record the initial global optimal solution $\mathbf{g}$.
while $FE < FE_{\max}$ **do**
  Compute the search progress $\tau = FE/FE_{\max}$.
  Determine the number of elite individuals L^t using Equation (16).
  Construct the elite center $\mathbf{E}^t$ using Equation (19) and (20).
  Generate the dynamic guide $\mathbf{G}^t$ using Equation (21).
  Compute the fractional memory position $\mathbf{x}_{\mathrm{mem},i}^t$ using Equation (13).
  Compute the alpha-coupled search factor r_1^t using Equation (22).
  Generate candidate individuals using Equation (26) and (27).
  Apply boundary handling using Equation (28).
  for $i = 1$ to N **do**
    Evaluate candidate individuals using Equation (6) and update FE.
    Apply greedy acceptance using Equation (29).
    Update the global optimal solution using Equation (30).
    Record the best-so-far fitness in $C(FE)$.
  end for
  Update the historical position caches.
end while
return Return the optimal solution $\mathbf{g}$, its fitness value $f(\mathbf{g})$ and its convergence curve C.

3.8. Computational Complexity Analysis

The computational complexity of the FOSCA mainly arises from elite sorting, fractional-memory position reconstruction, sine–cosine candidate generation, boundary handling and wrapper-based fitness evaluations. For one population update, sorting the population according to fitness values requires $O(N \log N)$, whereas fractional-memory reconstruction, dynamic guidance application, candidate generation and boundary handling require $O(ND)$, where N denotes the population size and D is the feature dimension. Therefore, without considering the cost of the wrapper objective function, the search-operator complexity of the FOSCA in one population update can be expressed as $O(ND + N \log N)$.

Let T denote the number of population updates before reaching $FE_{\max}$. The overall complexity of the internal search mechanism is then given by $O(T(ND + N \log N))$. Since each candidate solution must be evaluated by the wrapper fitness function, the total computational cost is more accurately formulated as

$$O\Big(T(ND + N \log N) + FE_{\max} C_f\Big), \tag{32}$$

where C_f denotes the computational cost of one wrapper-based fitness evaluation. In feature-selection tasks, C_f is usually dominated by classifier training and predictions during cross-validation. Therefore, although the dynamic elite-center strategy introduces an additional sorting operation, the major computational burden still comes from repeated evaluations of the wrapper objective function.

4. Numerical Results and Analysis on Benchmark Test Suites

This section evaluates the proposed FOSCA for solving the feature-selection problem. The details of the used performance measures, datasets, and compared algorithms and a discussion of the results are provided as follows:

4.1. Experimental Organization

We used 14 datasets to evaluate and compare the effectiveness of the proposed FOSCA. Their details are reported in Table 1; all datasets were obtained from the UCI Machine Learning Repository (https://archive.ics.uci.edu/ (accessed on 18 April 2026)). These datasets contain 18 to 7129 features, 50 to 5456 samples and two to 40 classes. They include low-dimensional datasets, medium-dimensional datasets, image datasets and high-dimensional biomedical datasets, providing a comprehensive benchmark for evaluating the generalization ability of the compared algorithms.

The proposed FOSCA was compared with 15 representative metaheuristic and feature-selection algorithms, including the MPA [56], HHO [23], PSO [19], SSA [57], GA [58], GWO [21], WOA [22], SCA [37], DE [20], LSHADE [59], JADE [60], OBSCA [26], M-SCA [25], MTVSCA [24], and FOPSO [31]. For each dataset, the samples were randomly split into training and testing subsets at a ratio of 7:3. The same partition was used for all algorithms and independent runs. All remaining configurations followed the original settings of each algorithm. The experiments were conducted on a 64-bit Windows 11 system with an AMD Ryzen 9 7940H processor running at 4.00 GHz with eight cores, 16 logical threads, and 16 GB of RAM. All of the algorithms presented in this paper were implemented in MATLAB R2025b, and the experimental results were visualized using MATLAB R2025b and Python 3.12.

Table 1. Details of datasets.

Dataset	Dataset Name	Features	Samples	Classes
Data 1	Vehicle	18	846	4
Data 2	GermanCredit	20	1000	2
Data 3	WallRobot	24	5456	4
Data 4	Ionosphere	34	351	2
Data 5	Promoters	57	106	2
Data 6	MUSK1	166	476	2
Data 7	Isolet	617	1560	26
Data 8	ORL	1024	400	40
Data 9	Colon	2000	62	2
Data 10	WarpPIE10P	2420	210	10
Data 11	GLIOMA	4434	50	4
Data 12	TOX-171	5748	171	4
Data 13	Leukemia	7070	72	2
Data 14	ALLAML	7129	72	2

In each independent run, each individual in the population encoded a candidate feature subset. Its quality was measured by a wrapper-based fitness function that combines the 5-fold cross-validation misclassification error of a KNN classifier on the training subset with the proportion of selected features. The fitness function is defined in Equation (33):

$$Fitness = 0.99 \times MCE_{CV} + 0.01 \times \frac{|S|}{D}, \tag{33}$$

where MCE_{CV} denotes the average misclassification error obtained by 5-fold cross-validation, $|S|$ is the number of selected features and D is the total number of original features. A lower fitness value indicates a better feature subset. After the optimization process was completed, the best feature subset obtained from the training subset was used to reconstruct both the complete training set and the testing set. Then, an external KNN classifier was trained on the reduced training set and evaluated on the reduced testing set.

For each algorithm–dataset pair, 30 independent runs were conducted. The population size was set to 30 and the maximum number of function evaluations was set to 10,000. The final performance was evaluated using the accuracy, precision, recall, F1-score and number of selected features.

4.2. Comparison of Performance Metrics

Tables 2–5 report the comparative results of the FOSCA and the other 15 algorithms on 14 datasets in terms of the accuracy, F1-score, precision, and recall. The FOSCA achieved the best mean accuracy, F1-score and precision on seven datasets and the best mean recall on six datasets, showing a strong competitiveness across classification metrics. Notably, the FOSCA obtained the best results on most metrics for DS2, DS3, DS4, DS5, DS7 and DS14, indicating that the selected feature subsets preserved discriminative information across datasets with diverse characteristics. Compared with the original SCA, the FOSCA improved all four metrics on 12 of the 14 datasets, confirming that the proposed mechanisms substantially enhance the search capability of the SCA for feature selection. The average ranking across the four metrics further shows that the FOSCA achieved the best overall rank among all of the compared algorithms, followed by the JADE, LSHADE, DE and SSA. The cross-metric ranking suggests that the FOSCA's advantage extends beyond one metric or a small subset of datasets.

Table 2. Comparison of the accuracy produced by the FOSCA and other algorithms.

Dataset	MPA		HHO		PSO		SSA		GA		GWO		WOA		SCA	
	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std
DS1	7.24×10^{-1}	1.02×10^{-2}	7.21×10^{-1}	1.54×10^{-2}	7.28×10^{-1}	1.13×10^{-2}	**7.31×10^{-1}**	1.52×10^{-2}	7.14×10^{-1}	2.19×10^{-2}	7.26×10^{-1}	9.87×10^{-3}	7.25×10^{-1}	1.32×10^{-2}	7.08×10^{-1}	1.60×10^{-2}
DS2	7.18×10^{-1}	1.89×10^{-2}	7.06×10^{-1}	1.80×10^{-2}	7.20×10^{-1}	2.10×10^{-2}	7.09×10^{-1}	1.74×10^{-2}	7.06×10^{-1}	2.18×10^{-2}	7.10×10^{-1}	1.79×10^{-2}	7.10×10^{-1}	2.09×10^{-2}	7.06×10^{-1}	1.91×10^{-2}
DS3	9.43×10^{-1}	1.65×10^{-3}	9.35×10^{-1}	8.61×10^{-3}	9.30×10^{-1}	9.81×10^{-3}	9.37×10^{-1}	8.51×10^{-3}	9.14×10^{-1}	1.91×10^{-2}	9.41×10^{-1}	5.50×10^{-3}	9.22×10^{-1}	9.06×10^{-3}	9.36×10^{-1}	7.79×10^{-3}
DS4	8.31×10^{-1}	2.91×10^{-2}	8.55×10^{-1}	2.70×10^{-2}	8.56×10^{-1}	2.70×10^{-2}	8.52×10^{-1}	2.74×10^{-2}	8.51×10^{-1}	2.20×10^{-2}	8.39×10^{-1}	3.16×10^{-2}	8.60×10^{-1}	2.08×10^{-2}	8.43×10^{-1}	2.29×10^{-2}
DS5	7.06×10^{-1}	6.30×10^{-2}	6.94×10^{-1}	7.66×10^{-2}	7.32×10^{-1}	7.39×10^{-2}	7.37×10^{-1}	7.04×10^{-2}	6.90×10^{-1}	7.56×10^{-2}	7.08×10^{-1}	6.45×10^{-2}	6.95×10^{-1}	6.92×10^{-2}	7.46×10^{-1}	7.66×10^{-2}
DS6	8.79×10^{-1}	2.34×10^{-2}	8.81×10^{-1}	2.06×10^{-2}	8.88×10^{-1}	1.41×10^{-2}	8.90×10^{-1}	2.13×10^{-2}	8.66×10^{-1}	2.36×10^{-2}	8.80×10^{-1}	2.39×10^{-2}	8.77×10^{-1}	2.13×10^{-2}	8.46×10^{-1}	3.67×10^{-2}
DS7	8.85×10^{-1}	1.25×10^{-2}	8.63×10^{-1}	1.31×10^{-2}	8.74×10^{-1}	1.04×10^{-2}	8.74×10^{-1}	1.36×10^{-2}	8.49×10^{-1}	1.15×10^{-2}	8.83×10^{-1}	1.49×10^{-2}	8.49×10^{-1}	1.15×10^{-2}	8.48×10^{-1}	1.52×10^{-2}
DS8	8.24×10^{-1}	2.54×10^{-2}	8.13×10^{-1}	2.24×10^{-2}	8.12×10^{-1}	1.84×10^{-2}	8.21×10^{-1}	2.89×10^{-2}	8.14×10^{-1}	2.06×10^{-2}	8.33×10^{-1}	2.24×10^{-2}	8.10×10^{-1}	2.02×10^{-2}	8.15×10^{-1}	2.53×10^{-2}
DS9	7.78×10^{-1}	7.29×10^{-2}	7.72×10^{-1}	6.08×10^{-2}	7.35×10^{-1}	3.77×10^{-2}	7.67×10^{-1}	5.54×10^{-2}	7.37×10^{-1}	5.04×10^{-2}	7.70×10^{-1}	4.78×10^{-2}	7.61×10^{-1}	6.71×10^{-2}	**7.94×10^{-1}**	6.87×10^{-2}
DS10	8.77×10^{-1}	1.30×10^{-2}	8.77×10^{-1}	1.08×10^{-2}	8.73×10^{-1}	5.90×10^{-3}	8.75×10^{-1}	4.84×10^{-3}	8.75×10^{-1}	8.05×10^{-3}	8.74×10^{-1}	9.26×10^{-3}	8.71×10^{-1}	9.98×10^{-3}	8.70×10^{-1}	2.74×10^{-2}
DS11	6.67×10^{-1}	7.22×10^{-2}	7.16×10^{-1}	6.54×10^{-2}	7.53×10^{-1}	5.30×10^{-2}	7.02×10^{-1}	4.87×10^{-2}	7.42×10^{-1}	4.54×10^{-2}	6.93×10^{-1}	5.13×10^{-2}	6.44×10^{-1}	8.98×10^{-2}	6.22×10^{-1}	8.64×10^{-2}
DS12	6.92×10^{-1}	3.89×10^{-2}	6.96×10^{-1}	3.25×10^{-2}	6.96×10^{-1}	3.70×10^{-2}	6.93×10^{-1}	3.19×10^{-2}	6.93×10^{-1}	2.80×10^{-2}	**7.66×10^{-1}**	3.85×10^{-2}	7.48×10^{-1}	3.88×10^{-2}	7.59×10^{-1}	4.55×10^{-2}
DS13	9.11×10^{-1}	4.28×10^{-2}	8.95×10^{-1}	5.79×10^{-2}	9.44×10^{-1}	2.20×10^{-2}	9.33×10^{-1}	2.96×10^{-2}	9.40×10^{-1}	2.14×10^{-2}	8.44×10^{-1}	3.74×10^{-2}	8.49×10^{-1}	2.53×10^{-2}	8.56×10^{-1}	6.31×10^{-2}
DS14	8.75×10^{-1}	4.92×10^{-2}	8.68×10^{-1}	8.27×10^{-2}	9.05×10^{-1}	3.54×10^{-2}	9.06×10^{-1}	3.85×10^{-2}	9.03×10^{-1}	3.18×10^{-2}	9.11×10^{-1}	3.90×10^{-2}	8.49×10^{-1}	5.74×10^{-2}	8.63×10^{-1}	7.04×10^{-2}

Dataset	DE		LSHADE		JADE		OBSCA		M-SCA		MTVSCA		FOPSO		FOSCA	
	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std
DS1	7.28×10^{-1}	6.39×10^{-3}	7.28×10^{-1}	9.63×10^{-3}	7.30×10^{-1}	9.12×10^{-3}	7.01×10^{-1}	2.80×10^{-2}	7.05×10^{-1}	2.41×10^{-2}	6.97×10^{-1}	1.95×10^{-2}	6.93×10^{-1}	1.91×10^{-2}	**7.31×10^{-1}**	5.08×10^{-3}
DS2	7.08×10^{-1}	1.49×10^{-2}	7.08×10^{-1}	1.25×10^{-2}	7.13×10^{-1}	1.32×10^{-2}	7.06×10^{-1}	2.71×10^{-2}	7.07×10^{-1}	2.68×10^{-2}	7.20×10^{-1}	2.01×10^{-2}	7.07×10^{-1}	1.50×10^{-2}	**7.68×10^{-1}**	5.92×10^{-3}
DS3	9.42×10^{-1}	1.34×10^{-3}	9.42×10^{-1}	2.14×10^{-3}	9.43×10^{-1}	1.05×10^{-3}	9.42×10^{-1}	6.98×10^{-3}	9.41×10^{-1}	6.44×10^{-3}	9.45×10^{-1}	3.64×10^{-3}	9.33×10^{-1}	8.22×10^{-3}	**9.54×10^{-1}**	5.65×10^{-16}
DS4	8.29×10^{-1}	2.62×10^{-2}	8.30×10^{-1}	2.77×10^{-2}	8.44×10^{-1}	2.69×10^{-2}	8.66×10^{-1}	2.17×10^{-2}	8.63×10^{-1}	2.34×10^{-2}	8.49×10^{-1}	2.63×10^{-2}	8.47×10^{-1}	2.08×10^{-2}	**9.19×10^{-1}**	9.27×10^{-3}
DS5	7.23×10^{-1}	6.03×10^{-2}	7.17×10^{-1}	6.38×10^{-2}	7.03×10^{-1}	6.64×10^{-2}	6.68×10^{-1}	8.35×10^{-2}	6.69×10^{-1}	6.88×10^{-2}	6.77×10^{-1}	7.53×10^{-2}	6.60×10^{-1}	7.12×10^{-2}	**7.57×10^{-1}**	4.30×10^{-2}
DS6	8.99×10^{-1}	2.29×10^{-2}	9.02×10^{-1}	1.74×10^{-2}	**9.06×10^{-1}**	1.67×10^{-2}	8.19×10^{-1}	2.35×10^{-2}	8.05×10^{-1}	2.29×10^{-2}	8.42×10^{-1}	1.52×10^{-2}	8.79×10^{-1}	2.18×10^{-2}	8.78×10^{-1}	1.31×10^{-2}
DS7	8.98×10^{-1}	7.87×10^{-3}	9.00×10^{-1}	1.15×10^{-2}	9.03×10^{-1}	8.65×10^{-3}	8.77×10^{-1}	1.09×10^{-2}	8.62×10^{-1}	1.71×10^{-2}	8.97×10^{-1}	8.49×10^{-3}	8.93×10^{-1}	1.25×10^{-2}	**9.04×10^{-1}**	6.91×10^{-3}
DS8	8.25×10^{-1}	2.09×10^{-2}	8.35×10^{-1}	2.01×10^{-2}	**8.46×10^{-1}**	1.89×10^{-2}	8.12×10^{-1}	2.61×10^{-2}	7.91×10^{-1}	3.23×10^{-2}	8.24×10^{-1}	2.63×10^{-2}	8.24×10^{-1}	1.90×10^{-2}	8.25×10^{-1}	1.13×10^{-2}
DS9	7.37×10^{-1}	5.64×10^{-2}	7.30×10^{-1}	4.55×10^{-2}	7.39×10^{-1}	3.90×10^{-2}	7.19×10^{-1}	6.35×10^{-2}	7.52×10^{-1}	6.80×10^{-2}	7.56×10^{-1}	5.94×10^{-2}	7.56×10^{-1}	4.75×10^{-2}	7.59×10^{-1}	4.46×10^{-2}
DS10	8.74×10^{-1}	2.90×10^{-3}	8.75×10^{-1}	4.84×10^{-3}	8.74×10^{-1}	4.03×10^{-3}	9.38×10^{-1}	8.69×10^{-3}	**9.41×10^{-1}**	1.77×10^{-2}	9.37×10^{-1}	2.90×10^{-3}	9.37×10^{-1}	2.90×10^{-3}	9.27×10^{-1}	8.94×10^{-3}
DS11	7.27×10^{-1}	4.41×10^{-2}	7.38×10^{-1}	4.61×10^{-2}	7.29×10^{-1}	4.93×10^{-2}	8.44×10^{-1}	8.09×10^{-2}	8.13×10^{-1}	1.07×10^{-1}	**8.56×10^{-1}**	5.28×10^{-2}	8.40×10^{-1}	5.13×10^{-2}	7.56×10^{-1}	3.64×10^{-2}
DS12	7.05×10^{-1}	2.60×10^{-2}	7.46×10^{-1}	2.33×10^{-2}	7.61×10^{-1}	2.25×10^{-2}	6.70×10^{-1}	3.72×10^{-2}	6.60×10^{-1}	4.41×10^{-2}	6.69×10^{-1}	1.81×10^{-2}	6.66×10^{-1}	2.03×10^{-2}	6.95×10^{-1}	1.12×10^{-2}
DS13	9.48×10^{-1}	1.45×10^{-2}	9.43×10^{-1}	1.94×10^{-2}	**9.49×10^{-1}**	1.21×10^{-2}	8.83×10^{-1}	5.12×10^{-2}	8.63×10^{-1}	6.82×10^{-2}	9.05×10^{-1}	1.25×10^{-2}	8.40×10^{-1}	2.65×10^{-2}	9.19×10^{-1}	2.22×10^{-2}
DS14	9.11×10^{-1}	3.90×10^{-2}	9.10×10^{-1}	3.61×10^{-2}	9.29×10^{-1}	3.25×10^{-2}	8.70×10^{-1}	4.32×10^{-2}	8.48×10^{-1}	6.55×10^{-2}	9.17×10^{-1}	4.32×10^{-2}	9.00×10^{-1}	3.39×10^{-2}	**9.33×10^{-1}**	2.37×10^{-2}

Table 3. Comparison of the F1-score values between the FOSCA and other algorithms.

Dataset	MPA		HHO		PSO		SSA		GA		GWO		WOA		SCA	
	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std
DS1	7.25×10^{-1}	1.02×10^{-2}	7.21×10^{-1}	1.58×10^{-2}	7.27×10^{-1}	1.26×10^{-2}	7.32×10^{-1}	1.62×10^{-2}	7.12×10^{-1}	2.29×10^{-2}	7.26×10^{-1}	1.10×10^{-2}	7.26×10^{-1}	1.31×10^{-2}	7.05×10^{-1}	1.69×10^{-2}
DS2	6.43×10^{-1}	2.74×10^{-2}	6.24×10^{-1}	2.33×10^{-2}	6.39×10^{-1}	2.77×10^{-2}	6.28×10^{-1}	2.13×10^{-2}	6.26×10^{-1}	2.95×10^{-2}	6.29×10^{-1}	2.78×10^{-2}	6.27×10^{-1}	2.54×10^{-2}	6.25×10^{-1}	3.16×10^{-2}
DS3	9.38×10^{-1}	1.93×10^{-3}	9.28×10^{-1}	1.15×10^{-2}	9.23×10^{-1}	1.18×10^{-2}	9.29×10^{-1}	1.37×10^{-2}	9.03×10^{-1}	2.06×10^{-2}	9.36×10^{-1}	6.53×10^{-3}	9.08×10^{-1}	1.40×10^{-2}	9.27×10^{-1}	1.16×10^{-2}
DS4	8.14×10^{-1}	3.16×10^{-2}	8.36×10^{-1}	3.02×10^{-2}	8.37×10^{-1}	2.96×10^{-2}	8.36×10^{-1}	3.06×10^{-2}	8.27×10^{-1}	2.82×10^{-2}	8.23×10^{-1}	3.52×10^{-2}	8.42×10^{-1}	2.48×10^{-2}	8.27×10^{-1}	2.55×10^{-2}
DS5	7.03×10^{-1}	6.52×10^{-2}	6.91×10^{-1}	7.87×10^{-2}	7.31×10^{-1}	7.50×10^{-2}	7.35×10^{-1}	7.11×10^{-2}	6.87×10^{-1}	7.79×10^{-2}	7.06×10^{-1}	6.47×10^{-2}	6.91×10^{-1}	7.22×10^{-2}	7.44×10^{-1}	7.76×10^{-2}
DS6	8.77×10^{-1}	2.41×10^{-2}	8.79×10^{-1}	2.08×10^{-2}	8.87×10^{-1}	1.41×10^{-2}	8.89×10^{-1}	2.14×10^{-2}	8.65×10^{-1}	2.36×10^{-2}	8.78×10^{-1}	2.47×10^{-2}	8.76×10^{-1}	2.18×10^{-2}	8.42×10^{-1}	3.75×10^{-2}
DS7	8.84×10^{-1}	1.28×10^{-2}	8.61×10^{-1}	1.35×10^{-2}	8.73×10^{-1}	1.05×10^{-2}	8.72×10^{-1}	1.42×10^{-2}	8.47×10^{-1}	1.17×10^{-2}	8.82×10^{-1}	1.48×10^{-2}	8.48×10^{-1}	1.14×10^{-2}	8.47×10^{-1}	1.53×10^{-2}
DS8	8.19×10^{-1}	2.88×10^{-2}	8.05×10^{-1}	2.42×10^{-2}	8.04×10^{-1}	2.10×10^{-2}	8.12×10^{-1}	3.32×10^{-2}	8.07×10^{-1}	2.18×10^{-2}	8.27×10^{-1}	2.48×10^{-2}	8.00×10^{-1}	2.36×10^{-2}	8.07×10^{-1}	2.66×10^{-2}
DS9	7.21×10^{-1}	9.66×10^{-2}	7.14×10^{-1}	8.85×10^{-2}	6.85×10^{-1}	5.38×10^{-2}	7.19×10^{-1}	6.83×10^{-2}	6.87×10^{-1}	6.89×10^{-2}	7.17×10^{-1}	7.44×10^{-2}	7.04×10^{-1}	8.89×10^{-2}	**7.45×10^{-1}**	9.50×10^{-2}
DS10	8.76×10^{-1}	1.31×10^{-2}	8.76×10^{-1}	1.07×10^{-2}	8.73×10^{-1}	6.08×10^{-3}	8.74×10^{-1}	4.71×10^{-3}	8.75×10^{-1}	7.71×10^{-3}	8.74×10^{-1}	8.89×10^{-3}	8.71×10^{-1}	9.45×10^{-3}	8.71×10^{-1}	2.65×10^{-2}
DS11	6.85×10^{-1}	8.06×10^{-2}	7.36×10^{-1}	8.94×10^{-2}	7.82×10^{-1}	6.27×10^{-2}	7.26×10^{-1}	5.98×10^{-2}	7.69×10^{-1}	5.68×10^{-2}	7.18×10^{-1}	5.70×10^{-2}	6.63×10^{-1}	1.18×10^{-1}	6.34×10^{-1}	1.03×10^{-1}
DS12	6.88×10^{-1}	3.97×10^{-2}	6.92×10^{-1}	3.61×10^{-2}	6.94×10^{-1}	3.74×10^{-2}	6.93×10^{-1}	3.27×10^{-2}	6.92×10^{-1}	2.75×10^{-2}	**7.66×10^{-1}**	3.90×10^{-2}	7.49×10^{-1}	3.77×10^{-2}	7.61×10^{-1}	4.44×10^{-2}
DS13	8.93×10^{-1}	5.30×10^{-2}	8.78×10^{-1}	6.38×10^{-2}	9.34×10^{-1}	2.91×10^{-2}	9.20×10^{-1}	3.77×10^{-2}	9.28×10^{-1}	2.74×10^{-2}	8.06×10^{-1}	4.63×10^{-2}	8.09×10^{-1}	3.18×10^{-2}	8.20×10^{-1}	8.33×10^{-2}
DS14	8.51×10^{-1}	5.94×10^{-2}	8.39×10^{-1}	1.09×10^{-1}	8.84×10^{-1}	4.42×10^{-2}	8.86×10^{-1}	4.88×10^{-2}	8.82×10^{-1}	4.10×10^{-2}	8.94×10^{-1}	4.64×10^{-2}	8.21×10^{-1}	6.52×10^{-2}	8.35×10^{-1}	8.73×10^{-2}

Dataset	DE		LSHADE		JADE		OBSCA		M-SCA		MTVSCA		FOPSO		FOSCA	
	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std
DS1	7.31×10^{-1}	6.67×10^{-3}	7.30×10^{-1}	1.07×10^{-2}	7.32×10^{-1}	9.39×10^{-3}	6.96×10^{-1}	2.89×10^{-2}	7.01×10^{-1}	2.49×10^{-2}	6.92×10^{-1}	1.98×10^{-2}	6.88×10^{-1}	1.96×10^{-2}	**7.33×10^{-1}**	5.55×10^{-3}
DS2	6.26×10^{-1}	1.94×10^{-2}	6.25×10^{-1}	1.62×10^{-2}	6.33×10^{-1}	2.22×10^{-2}	6.21×10^{-1}	3.80×10^{-2}	6.25×10^{-1}	4.20×10^{-2}	6.45×10^{-1}	3.22×10^{-2}	6.24×10^{-1}	2.17×10^{-2}	**6.80×10^{-1}**	1.09×10^{-2}
DS3	9.38×10^{-1}	1.81×10^{-3}	9.37×10^{-1}	3.06×10^{-3}	9.38×10^{-1}	5.86×10^{-4}	9.33×10^{-1}	1.31×10^{-2}	9.32×10^{-1}	9.89×10^{-3}	9.38×10^{-1}	5.47×10^{-3}	9.26×10^{-1}	9.63×10^{-3}	**9.52×10^{-1}**	5.65×10^{-16}
DS4	8.12×10^{-1}	2.86×10^{-2}	8.14×10^{-1}	3.03×10^{-2}	8.28×10^{-1}	3.07×10^{-2}	8.49×10^{-1}	2.67×10^{-2}	8.45×10^{-1}	2.76×10^{-2}	8.27×10^{-1}	3.09×10^{-2}	8.20×10^{-1}	2.70×10^{-2}	**9.09×10^{-1}**	1.11×10^{-2}
DS5	7.22×10^{-1}	6.08×10^{-2}	7.16×10^{-1}	6.39×10^{-2}	7.02×10^{-1}	6.64×10^{-2}	6.65×10^{-1}	8.44×10^{-2}	6.65×10^{-1}	7.13×10^{-2}	6.75×10^{-1}	7.51×10^{-2}	6.56×10^{-1}	7.37×10^{-2}	**7.55×10^{-1}**	4.33×10^{-2}
DS6	8.97×10^{-1}	2.32×10^{-2}	9.01×10^{-1}	1.76×10^{-2}	**9.05×10^{-1}**	1.69×10^{-2}	8.16×10^{-1}	2.34×10^{-2}	8.03×10^{-1}	2.33×10^{-2}	8.40×10^{-1}	1.51×10^{-2}	8.77×10^{-1}	2.25×10^{-2}	8.78×10^{-1}	1.31×10^{-2}
DS7	8.98×10^{-1}	7.88×10^{-3}	9.00×10^{-1}	1.17×10^{-2}	9.02×10^{-1}	8.82×10^{-3}	8.76×10^{-1}	1.12×10^{-2}	8.61×10^{-1}	1.72×10^{-2}	8.97×10^{-1}	8.55×10^{-3}	8.92×10^{-1}	1.25×10^{-2}	**9.04×10^{-1}**	6.82×10^{-3}
DS8	8.19×10^{-1}	2.36×10^{-2}	8.29×10^{-1}	2.14×10^{-2}	**8.42×10^{-1}**	2.05×10^{-2}	8.05×10^{-1}	2.76×10^{-2}	7.84×10^{-1}	3.51×10^{-2}	8.16×10^{-1}	2.59×10^{-2}	8.15×10^{-1}	1.98×10^{-2}	8.12×10^{-1}	1.60×10^{-2}
DS9	6.83×10^{-1}	7.45×10^{-2}	6.74×10^{-1}	6.66×10^{-2}	6.90×10^{-1}	5.47×10^{-2}	6.74×10^{-1}	6.85×10^{-2}	7.09×10^{-1}	8.64×10^{-2}	7.17×10^{-1}	6.76×10^{-2}	7.15×10^{-1}	5.16×10^{-2}	7.11×10^{-1}	5.66×10^{-2}
DS10	8.73×10^{-1}	2.69×10^{-3}	8.74×10^{-1}	4.82×10^{-3}	8.74×10^{-1}	3.60×10^{-3}	9.33×10^{-1}	9.68×10^{-3}	**9.36×10^{-1}**	1.91×10^{-2}	9.31×10^{-1}	3.32×10^{-3}	9.31×10^{-1}	3.40×10^{-3}	9.26×10^{-1}	9.21×10^{-3}
DS11	7.60×10^{-1}	5.55×10^{-2}	7.61×10^{-1}	5.91×10^{-2}	7.59×10^{-1}	6.12×10^{-2}	8.44×10^{-1}	1.03×10^{-1}	8.13×10^{-1}	1.14×10^{-1}	**8.71×10^{-1}**	4.70×10^{-2}	8.58×10^{-1}	4.48×10^{-2}	7.42×10^{-1}	4.39×10^{-2}
DS12	7.03×10^{-1}	2.67×10^{-2}	7.49×10^{-1}	2.14×10^{-2}	7.63×10^{-1}	2.04×10^{-2}	6.77×10^{-1}	3.56×10^{-2}	6.69×10^{-1}	4.18×10^{-2}	6.76×10^{-1}	1.74×10^{-2}	6.74×10^{-1}	1.96×10^{-2}	7.05×10^{-1}	1.19×10^{-2}
DS13	9.38×10^{-1}	1.86×10^{-2}	9.32×10^{-1}	2.48×10^{-2}	**9.40×10^{-1}**	1.55×10^{-2}	8.53×10^{-1}	6.73×10^{-2}	8.29×10^{-1}	9.30×10^{-2}	8.83×10^{-1}	1.70×10^{-2}	7.98×10^{-1}	2.88×10^{-2}	9.02×10^{-1}	2.81×10^{-2}
DS14	8.92×10^{-1}	4.96×10^{-2}	8.90×10^{-1}	4.71×10^{-2}	9.14×10^{-1}	4.18×10^{-2}	8.43×10^{-1}	5.65×10^{-2}	8.16×10^{-1}	8.17×10^{-2}	8.99×10^{-1}	5.61×10^{-2}	8.78×10^{-1}	4.36×10^{-2}	**9.21×10^{-1}**	2.93×10^{-2}

Table 4. Comparison of the precision values between the FOSCA and other algorithms.

Dataset	MPA		HHO		PSO		SSA		GA		GWO		WOA		SCA	
	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std
DS1	7.25×10^{-1}	1.13×10^{-2}	7.19×10^{-1}	1.68×10^{-2}	7.25×10^{-1}	1.36×10^{-2}	7.30×10^{-1}	1.70×10^{-2}	7.09×10^{-1}	2.38×10^{-2}	7.25×10^{-1}	1.28×10^{-2}	7.24×10^{-1}	1.35×10^{-2}	7.02×10^{-1}	1.73×10^{-2}
DS2	6.56×10^{-1}	2.56×10^{-2}	6.40×10^{-1}	2.50×10^{-2}	6.58×10^{-1}	2.97×10^{-2}	6.43×10^{-1}	2.35×10^{-2}	6.40×10^{-1}	3.04×10^{-2}	6.45×10^{-1}	2.52×10^{-2}	6.44×10^{-1}	2.85×10^{-2}	6.40×10^{-1}	2.80×10^{-2}
DS3	9.42×10^{-1}	2.64×10^{-3}	9.32×10^{-1}	1.26×10^{-2}	9.28×10^{-1}	1.26×10^{-2}	9.34×10^{-1}	1.42×10^{-2}	9.07×10^{-1}	2.01×10^{-2}	9.40×10^{-1}	7.77×10^{-3}	9.10×10^{-1}	1.56×10^{-2}	9.30×10^{-1}	1.33×10^{-2}
DS4	8.21×10^{-1}	3.40×10^{-2}	8.57×10^{-1}	3.47×10^{-2}	8.63×10^{-1}	3.59×10^{-2}	8.50×10^{-1}	3.36×10^{-2}	8.69×10^{-1}	2.09×10^{-2}	8.29×10^{-1}	3.54×10^{-2}	8.61×10^{-1}	2.30×10^{-2}	8.37×10^{-1}	2.69×10^{-2}
DS5	7.13×10^{-1}	6.33×10^{-2}	6.98×10^{-1}	7.73×10^{-2}	7.37×10^{-1}	7.51×10^{-2}	7.42×10^{-1}	7.29×10^{-2}	6.95×10^{-1}	7.75×10^{-2}	7.12×10^{-1}	6.73×10^{-2}	6.99×10^{-1}	7.01×10^{-2}	7.51×10^{-1}	7.96×10^{-2}
DS6	8.78×10^{-1}	2.33×10^{-2}	8.80×10^{-1}	2.08×10^{-2}	8.87×10^{-1}	1.39×10^{-2}	8.89×10^{-1}	2.12×10^{-2}	8.66×10^{-1}	2.26×10^{-2}	8.80×10^{-1}	2.41×10^{-2}	8.76×10^{-1}	2.12×10^{-2}	8.46×10^{-1}	3.78×10^{-2}
DS7	8.95×10^{-1}	1.14×10^{-2}	8.75×10^{-1}	1.22×10^{-2}	8.85×10^{-1}	9.88×10^{-3}	8.85×10^{-1}	1.33×10^{-2}	8.60×10^{-1}	1.23×10^{-2}	8.89×10^{-1}	1.41×10^{-2}	8.58×10^{-1}	1.04×10^{-2}	8.56×10^{-1}	1.47×10^{-2}
DS8	8.69×10^{-1}	2.90×10^{-2}	8.58×10^{-1}	2.53×10^{-2}	8.54×10^{-1}	2.38×10^{-2}	8.62×10^{-1}	3.28×10^{-2}	8.61×10^{-1}	2.47×10^{-2}	8.69×10^{-1}	2.27×10^{-2}	8.49×10^{-1}	2.65×10^{-2}	8.51×10^{-1}	2.52×10^{-2}
DS9	7.78×10^{-1}	1.10×10^{-1}	7.63×10^{-1}	8.55×10^{-2}	7.01×10^{-1}	4.80×10^{-2}	7.53×10^{-1}	8.64×10^{-2}	7.01×10^{-1}	6.26×10^{-2}	7.54×10^{-1}	6.38×10^{-2}	7.49×10^{-1}	9.88×10^{-2}	$\mathbf{7.90 \times 10^{-1}}$	1.04×10^{-1}
DS10	9.01×10^{-1}	1.40×10^{-2}	9.04×10^{-1}	7.93×10^{-3}	9.05×10^{-1}	6.22×10^{-3}	9.05×10^{-1}	4.19×10^{-3}	9.03×10^{-1}	8.17×10^{-3}	9.02×10^{-1}	1.16×10^{-2}	8.98×10^{-1}	1.04×10^{-2}	8.91×10^{-1}	2.70×10^{-2}
DS11	7.37×10^{-1}	7.85×10^{-2}	7.76×10^{-1}	8.44×10^{-2}	8.14×10^{-1}	4.60×10^{-2}	7.72×10^{-1}	4.47×10^{-2}	8.05×10^{-1}	3.96×10^{-2}	7.63×10^{-1}	4.28×10^{-2}	6.92×10^{-1}	1.31×10^{-1}	6.83×10^{-1}	1.09×10^{-1}
DS12	7.32×10^{-1}	3.50×10^{-2}	7.35×10^{-1}	3.58×10^{-2}	7.40×10^{-1}	3.22×10^{-2}	7.43×10^{-1}	2.87×10^{-2}	7.38×10^{-1}	2.55×10^{-2}	7.90×10^{-1}	3.59×10^{-2}	7.73×10^{-1}	3.72×10^{-2}	7.84×10^{-1}	4.39×10^{-2}
DS13	9.27×10^{-1}	4.79×10^{-2}	9.06×10^{-1}	7.25×10^{-2}	9.62×10^{-1}	1.30×10^{-2}	9.53×10^{-1}	2.49×10^{-2}	9.59×10^{-1}	1.31×10^{-2}	8.68×10^{-1}	5.87×10^{-2}	8.85×10^{-1}	4.52×10^{-2}	8.74×10^{-1}	7.81×10^{-2}
DS14	8.81×10^{-1}	6.24×10^{-2}	8.78×10^{-1}	1.01×10^{-1}	9.28×10^{-1}	4.01×10^{-2}	9.28×10^{-1}	4.18×10^{-2}	9.30×10^{-1}	3.38×10^{-2}	9.26×10^{-1}	4.73×10^{-2}	8.57×10^{-1}	8.02×10^{-2}	8.77×10^{-1}	8.53×10^{-2}

Dataset	DE		LSHADE		JADE		OBSCA		M-SCA		MTVSCA		FOPSO		FOSCA	
	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std
DS1	7.31×10^{-1}	7.39×10^{-3}	7.30×10^{-1}	1.21×10^{-2}	7.32×10^{-1}	1.02×10^{-2}	6.90×10^{-1}	2.96×10^{-2}	6.96×10^{-1}	2.54×10^{-2}	6.86×10^{-1}	1.99×10^{-2}	6.82×10^{-1}	1.99×10^{-2}	$\mathbf{7.34 \times 10^{-1}}$	6.72×10^{-3}
DS2	6.42×10^{-1}	2.04×10^{-2}	6.42×10^{-1}	1.71×10^{-2}	6.49×10^{-1}	1.87×10^{-2}	6.38×10^{-1}	3.83×10^{-2}	6.40×10^{-1}	3.87×10^{-2}	6.59×10^{-1}	2.77×10^{-2}	6.40×10^{-1}	2.17×10^{-2}	$\mathbf{7.39 \times 10^{-1}}$	8.99×10^{-3}
DS3	9.41×10^{-1}	2.00×10^{-3}	9.42×10^{-1}	2.92×10^{-3}	9.41×10^{-1}	2.03×10^{-3}	9.35×10^{-1}	1.54×10^{-2}	9.34×10^{-1}	1.27×10^{-2}	9.39×10^{-1}	8.21×10^{-3}	9.25×10^{-1}	1.23×10^{-2}	$\mathbf{9.59 \times 10^{-1}}$	3.39×10^{-16}
DS4	8.17×10^{-1}	3.01×10^{-2}	8.19×10^{-1}	3.12×10^{-2}	8.34×10^{-1}	2.89×10^{-2}	8.62×10^{-1}	2.18×10^{-2}	8.58×10^{-1}	2.60×10^{-2}	8.46×10^{-1}	3.12×10^{-2}	8.57×10^{-1}	2.06×10^{-2}	$\mathbf{9.22 \times 10^{-1}}$	9.93×10^{-3}
DS5	7.26×10^{-1}	6.17×10^{-2}	7.20×10^{-1}	6.48×10^{-2}	7.06×10^{-1}	6.90×10^{-2}	6.71×10^{-1}	8.52×10^{-2}	6.74×10^{-1}	6.67×10^{-2}	6.82×10^{-1}	7.78×10^{-2}	6.67×10^{-1}	7.59×10^{-2}	$\mathbf{7.62 \times 10^{-1}}$	4.50×10^{-2}
DS6	8.98×10^{-1}	2.34×10^{-2}	9.01×10^{-1}	1.76×10^{-2}	$\mathbf{9.06 \times 10^{-1}}$	1.69×10^{-2}	8.17×10^{-1}	2.42×10^{-2}	8.04×10^{-1}	2.29×10^{-2}	8.40×10^{-1}	1.52×10^{-2}	8.77×10^{-1}	2.17×10^{-2}	8.83×10^{-1}	1.34×10^{-2}
DS7	9.04×10^{-1}	7.42×10^{-3}	9.06×10^{-1}	1.07×10^{-2}	9.08×10^{-1}	7.92×10^{-3}	8.87×10^{-1}	1.10×10^{-2}	8.72×10^{-1}	1.50×10^{-2}	9.05×10^{-1}	7.84×10^{-3}	9.00×10^{-1}	1.14×10^{-2}	$\mathbf{9.10 \times 10^{-1}}$	7.13×10^{-3}
DS8	8.67×10^{-1}	2.34×10^{-2}	8.77×10^{-1}	2.00×10^{-2}	$\mathbf{8.89 \times 10^{-1}}$	2.28×10^{-2}	8.44×10^{-1}	2.91×10^{-2}	8.29×10^{-1}	3.69×10^{-2}	8.54×10^{-1}	2.18×10^{-2}	8.52×10^{-1}	2.13×10^{-2}	8.59×10^{-1}	2.39×10^{-2}
DS9	7.03×10^{-1}	7.63×10^{-2}	6.93×10^{-1}	6.16×10^{-2}	7.05×10^{-1}	5.00×10^{-2}	6.87×10^{-1}	6.88×10^{-2}	7.28×10^{-1}	9.32×10^{-2}	7.28×10^{-1}	7.00×10^{-2}	7.31×10^{-1}	5.61×10^{-2}	7.35×10^{-1}	5.55×10^{-2}
DS10	9.03×10^{-1}	5.32×10^{-3}	9.04×10^{-1}	6.73×10^{-3}	9.03×10^{-1}	5.81×10^{-3}	9.43×10^{-1}	8.65×10^{-3}	$\mathbf{9.47 \times 10^{-1}}$	1.60×10^{-2}	9.41×10^{-1}	3.65×10^{-3}	9.41×10^{-1}	2.61×10^{-3}	9.41×10^{-1}	7.24×10^{-3}
DS11	7.94×10^{-1}	3.86×10^{-2}	8.02×10^{-1}	4.00×10^{-2}	7.94×10^{-1}	4.27×10^{-2}	8.79×10^{-1}	1.05×10^{-1}	8.52×10^{-1}	1.09×10^{-1}	$\mathbf{8.87 \times 10^{-1}}$	5.02×10^{-2}	8.69×10^{-1}	5.03×10^{-2}	8.11×10^{-1}	4.41×10^{-2}
DS12	7.48×10^{-1}	2.39×10^{-2}	7.73×10^{-1}	2.15×10^{-2}	$\mathbf{7.91 \times 10^{-1}}$	2.22×10^{-2}	6.91×10^{-1}	3.54×10^{-2}	6.94×10^{-1}	3.95×10^{-2}	6.89×10^{-1}	1.88×10^{-2}	6.90×10^{-1}	2.17×10^{-2}	7.40×10^{-1}	1.40×10^{-2}
DS13	9.64×10^{-1}	8.90×10^{-3}	9.61×10^{-1}	1.19×10^{-2}	$\mathbf{9.65 \times 10^{-1}}$	7.40×10^{-3}	9.15×10^{-1}	5.03×10^{-2}	8.85×10^{-1}	7.33×10^{-2}	9.38×10^{-1}	7.22×10^{-3}	8.70×10^{-1}	5.35×10^{-2}	9.43×10^{-1}	1.92×10^{-2}
DS14	9.34×10^{-1}	3.96×10^{-2}	9.34×10^{-1}	3.26×10^{-2}	9.49×10^{-1}	2.78×10^{-2}	8.83×10^{-1}	5.25×10^{-2}	8.54×10^{-1}	7.93×10^{-2}	9.40×10^{-1}	3.88×10^{-2}	9.24×10^{-1}	3.61×10^{-2}	$\mathbf{9.51 \times 10^{-1}}$	2.36×10^{-2}

Table 5. Comparison of the recall values between the FOSCA and other algorithms.

Dataset	MPA		HHO		PSO		SSA		GA		GWO		WOA		SCA	
	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std
DS1	7.28×10^{-1}	1.00×10^{-2}	7.24×10^{-1}	1.54×10^{-2}	7.31×10^{-1}	1.13×10^{-2}	$\mathbf{7.35 \times 10^{-1}}$	1.51×10^{-2}	7.17×10^{-1}	2.18×10^{-2}	7.30×10^{-1}	9.74×10^{-3}	7.29×10^{-1}	1.31×10^{-2}	7.12×10^{-1}	1.61×10^{-2}
DS2	6.37×10^{-1}	2.78×10^{-2}	6.18×10^{-1}	2.19×10^{-2}	6.32×10^{-1}	2.58×10^{-2}	6.22×10^{-1}	2.00×10^{-2}	6.21×10^{-1}	2.82×10^{-2}	6.24×10^{-1}	2.82×10^{-2}	6.21×10^{-1}	2.38×10^{-2}	6.21×10^{-1}	2.96×10^{-2}
DS3	9.34×10^{-1}	2.59×10^{-3}	9.25×10^{-1}	1.11×10^{-2}	9.20×10^{-1}	1.18×10^{-2}	9.25×10^{-1}	1.37×10^{-2}	9.00×10^{-1}	2.19×10^{-2}	9.32×10^{-1}	5.81×10^{-3}	9.07×10^{-1}	1.38×10^{-2}	9.25×10^{-1}	1.08×10^{-2}
DS4	8.09×10^{-1}	3.06×10^{-2}	8.26×10^{-1}	2.96×10^{-2}	8.24×10^{-1}	2.79×10^{-2}	8.28×10^{-1}	3.05×10^{-2}	8.10×10^{-1}	2.89×10^{-2}	8.19×10^{-1}	3.51×10^{-2}	8.33×10^{-1}	2.68×10^{-2}	8.20×10^{-1}	2.56×10^{-2}
DS5	7.06×10^{-1}	6.35×10^{-2}	6.93×10^{-1}	7.77×10^{-2}	7.33×10^{-1}	7.47×10^{-2}	7.36×10^{-1}	7.13×10^{-2}	6.90×10^{-1}	7.69×10^{-2}	7.07×10^{-1}	6.42×10^{-2}	6.93×10^{-1}	7.10×10^{-2}	7.45×10^{-1}	7.65×10^{-2}
DS6	8.78×10^{-1}	2.52×10^{-2}	8.83×10^{-1}	2.15×10^{-2}	8.92×10^{-1}	1.41×10^{-2}	8.92×10^{-1}	2.15×10^{-2}	8.71×10^{-1}	2.34×10^{-2}	8.76×10^{-1}	2.54×10^{-2}	8.80×10^{-1}	2.31×10^{-2}	8.41×10^{-1}	3.76×10^{-2}
DS7	8.85×10^{-1}	1.25×10^{-2}	8.63×10^{-1}	1.31×10^{-2}	8.74×10^{-1}	1.04×10^{-2}	8.74×10^{-1}	1.36×10^{-2}	8.49×10^{-1}	1.15×10^{-2}	8.83×10^{-1}	1.49×10^{-2}	8.49×10^{-1}	1.15×10^{-2}	8.48×10^{-1}	1.52×10^{-2}
DS8	8.24×10^{-1}	2.54×10^{-2}	8.13×10^{-1}	2.24×10^{-2}	8.12×10^{-1}	1.84×10^{-2}	8.21×10^{-1}	2.89×10^{-2}	8.14×10^{-1}	2.06×10^{-2}	8.33×10^{-1}	2.24×10^{-2}	8.10×10^{-1}	2.02×10^{-2}	8.15×10^{-1}	2.53×10^{-2}
DS9	7.10×10^{-1}	8.65×10^{-2}	7.07×10^{-1}	8.37×10^{-2}	6.81×10^{-1}	5.40×10^{-2}	7.11×10^{-1}	6.56×10^{-2}	6.83×10^{-1}	7.06×10^{-2}	7.11×10^{-1}	6.91×10^{-2}	6.96×10^{-1}	8.05×10^{-2}	$\mathbf{7.35 \times 10^{-1}}$	8.58×10^{-2}
DS10	8.84×10^{-1}	1.28×10^{-2}	8.85×10^{-1}	1.04×10^{-2}	8.81×10^{-1}	5.98×10^{-3}	8.83×10^{-1}	5.09×10^{-3}	8.83×10^{-1}	7.88×10^{-3}	8.82×10^{-1}	9.03×10^{-3}	8.79×10^{-1}	1.02×10^{-2}	8.78×10^{-1}	2.54×10^{-2}
DS11	6.88×10^{-1}	8.62×10^{-2}	7.39×10^{-1}	8.80×10^{-2}	7.79×10^{-1}	7.33×10^{-2}	7.25×10^{-1}	7.04×10^{-2}	7.65×10^{-1}	6.59×10^{-2}	7.18×10^{-1}	6.27×10^{-2}	6.75×10^{-1}	1.06×10^{-1}	6.47×10^{-1}	1.03×10^{-1}
DS12	6.85×10^{-1}	3.91×10^{-2}	6.89×10^{-1}	3.30×10^{-2}	6.88×10^{-1}	3.73×10^{-2}	6.86×10^{-1}	3.23×10^{-2}	6.86×10^{-1}	2.79×10^{-2}	$\mathbf{7.65 \times 10^{-1}}$	3.91×10^{-2}	7.47×10^{-1}	3.95×10^{-2}	7.58×10^{-1}	4.51×10^{-2}
DS13	8.76×10^{-1}	5.61×10^{-2}	8.65×10^{-1}	5.98×10^{-2}	9.17×10^{-1}	3.29×10^{-2}	9.01×10^{-1}	4.16×10^{-2}	9.10×10^{-1}	3.21×10^{-2}	7.83×10^{-1}	4.29×10^{-2}	7.85×10^{-1}	3.04×10^{-2}	8.04×10^{-1}	8.48×10^{-2}
DS14	8.37×10^{-1}	5.98×10^{-2}	8.27×10^{-1}	1.03×10^{-1}	8.63×10^{-1}	4.60×10^{-2}	8.67×10^{-1}	5.30×10^{-2}	8.60×10^{-1}	4.39×10^{-2}	8.77×10^{-1}	4.66×10^{-2}	8.06×10^{-1}	6.13×10^{-2}	8.23×10^{-1}	9.02×10^{-2}

Dataset	DE		LSHADE		JADE		OBSCA		M-SCA		MTVSCA		FOPSO		FOSCA	
	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std	Avg.	Std
DS1	7.32×10^{-1}	6.29×10^{-3}	7.32×10^{-1}	9.48×10^{-3}	7.34×10^{-1}	9.10×10^{-3}	7.05×10^{-1}	2.77×10^{-2}	7.09×10^{-1}	2.42×10^{-2}	7.02×10^{-1}	1.96×10^{-2}	6.97×10^{-1}	1.94×10^{-2}	7.35×10^{-1}	5.02×10^{-3}
DS2	6.20×10^{-1}	1.85×10^{-2}	6.19×10^{-1}	1.55×10^{-2}	6.27×10^{-1}	2.28×10^{-2}	6.16×10^{-1}	3.56×10^{-2}	6.21×10^{-1}	4.06×10^{-2}	6.39×10^{-1}	3.39×10^{-2}	6.19×10^{-1}	2.06×10^{-2}	$\mathbf{6.64 \times 10^{-1}}$	9.83×10^{-3}
DS3	9.35×10^{-1}	2.61×10^{-3}	9.34×10^{-1}	4.13×10^{-3}	9.35×10^{-1}	8.91×10^{-4}	9.31×10^{-1}	1.17×10^{-2}	9.31×10^{-1}	8.42×10^{-3}	9.37×10^{-1}	2.78×10^{-3}	9.28×10^{-1}	9.01×10^{-3}	$\mathbf{9.45 \times 10^{-1}}$	6.78×10^{-16}
DS4	8.08×10^{-1}	2.79×10^{-2}	8.11×10^{-1}	2.97×10^{-2}	8.25×10^{-1}	3.19×10^{-2}	8.42×10^{-1}	3.11×10^{-2}	8.38×10^{-1}	3.01×10^{-2}	8.17×10^{-1}	3.26×10^{-2}	8.04×10^{-1}	2.85×10^{-2}	$\mathbf{9.00 \times 10^{-1}}$	1.38×10^{-2}
DS5	7.23×10^{-1}	6.12×10^{-2}	7.17×10^{-1}	6.37×10^{-2}	7.03×10^{-1}	6.66×10^{-2}	6.67×10^{-1}	8.39×10^{-2}	6.69×10^{-1}	6.83×10^{-2}	6.77×10^{-1}	7.52×10^{-2}	6.60×10^{-1}	7.26×10^{-2}	$\mathbf{7.56 \times 10^{-1}}$	4.34×10^{-2}
DS6	8.98×10^{-1}	2.34×10^{-2}	9.02×10^{-1}	1.78×10^{-2}	$\mathbf{9.05 \times 10^{-1}}$	1.69×10^{-2}	8.18×10^{-1}	2.28×10^{-2}	8.06×10^{-1}	2.38×10^{-2}	8.43×10^{-1}	1.49×10^{-2}	8.80×10^{-1}	2.38×10^{-2}	8.88×10^{-1}	1.34×10^{-2}
DS7	8.98×10^{-1}	7.87×10^{-3}	9.00×10^{-1}	1.15×10^{-2}	9.03×10^{-1}	8.65×10^{-3}	8.77×10^{-1}	1.09×10^{-2}	8.62×10^{-1}	1.71×10^{-2}	8.97×10^{-1}	8.49×10^{-3}	8.93×10^{-1}	1.25×10^{-2}	$\mathbf{9.04 \times 10^{-1}}$	6.91×10^{-3}
DS8	8.25×10^{-1}	2.09×10^{-2}	8.35×10^{-1}	2.01×10^{-2}	$\mathbf{8.46 \times 10^{-1}}$	1.89×10^{-2}	8.12×10^{-1}	2.61×10^{-2}	7.91×10^{-1}	3.23×10^{-2}	8.24×10^{-1}	2.63×10^{-2}	8.24×10^{-1}	1.90×10^{-2}	8.25×10^{-1}	1.13×10^{-2}
DS9	6.76×10^{-1}	7.06×10^{-2}	6.69×10^{-1}	6.47×10^{-2}	6.86×10^{-1}	5.66×10^{-2}	6.72×10^{-1}	6.72×10^{-2}	7.08×10^{-1}	8.48×10^{-2}	7.12×10^{-1}	6.86×10^{-2}	7.10×10^{-1}	5.19×10^{-2}	7.03×10^{-1}	5.44×10^{-2}
DS10	8.82×10^{-1}	3.04×10^{-3}	8.83×10^{-1}	4.86×10^{-3}	8.82×10^{-1}	4.23×10^{-3}	9.35×10^{-1}	8.97×10^{-3}	$\mathbf{9.38 \times 10^{-1}}$	1.84×10^{-2}	9.34×10^{-1}	3.04×10^{-3}	9.34×10^{-1}	3.04×10^{-3}	9.30×10^{-1}	8.19×10^{-3}
DS11	7.56×10^{-1}	6.39×10^{-2}	7.58×10^{-1}	6.78×10^{-2}	7.52×10^{-1}	6.96×10^{-2}	8.45×10^{-1}	9.68×10^{-2}	8.20×10^{-1}	1.05×10^{-1}	$\mathbf{8.68 \times 10^{-1}}$	4.56×10^{-2}	8.56×10^{-1}	4.30×10^{-2}	7.84×10^{-1}	3.05×10^{-2}
DS12	6.98×10^{-1}	2.65×10^{-2}	7.46×10^{-1}	2.34×10^{-2}	7.62×10^{-1}	2.23×10^{-2}	6.71×10^{-1}	3.68×10^{-2}	6.62×10^{-1}	4.33×10^{-2}	6.70×10^{-1}	1.73×10^{-2}	6.67×10^{-1}	1.95×10^{-2}	7.07×10^{-1}	9.78×10^{-3}
DS13	9.21×10^{-1}	2.18×10^{-2}	9.14×10^{-1}	2.91×10^{-2}	$\mathbf{9.24 \times 10^{-1}}$	1.81×10^{-2}	8.32×10^{-1}	7.27×10^{-2}	8.17×10^{-1}	9.46×10^{-2}	8.57×10^{-1}	1.88×10^{-2}	7.74×10^{-1}	2.36×10^{-2}	8.81×10^{-1}	3.29×10^{-2}
DS14	8.71×10^{-1}	5.27×10^{-2}	8.69×10^{-1}	5.17×10^{-2}	8.95×10^{-1}	4.59×10^{-2}	8.31×10^{-1}	6.00×10^{-2}	8.05×10^{-1}	8.27×10^{-2}	8.80×10^{-1}	6.04×10^{-2}	8.57×10^{-1}	4.69×10^{-2}	$\mathbf{9.04 \times 10^{-1}}$	3.27×10^{-2}

The standard deviations further demonstrate the stability of the FOSCA over 30 independent runs. The FOSCA achieved the lowest standard deviation on 11 datasets for accuracy, 11 datasets for recall, nine datasets for precision and 11 datasets for the F1-score. Its average standard deviation was also the smallest among all of the algorithms across the four classification metrics. Across repeated runs, the FOSCA combined a higher mean classification performance with lower variability. This improvement in both the performance and stability can be attributed to the combined effect of short-memory fractional-order reconstruction, dynamic elite-center guidance, a fast-decaying search amplitude and the

greedy individual-level acceptance strategy. Together, these strategies exploit historical search information, reduce premature dependence on a single best solution and suppress late-stage oscillations. Tables 2–5 therefore support the effectiveness and reliability of the FOSCA relative to the evaluated metaheuristic baselines.

4.3. Stability and Robustness Analysis

To analyze the stability of different algorithms across independent runs, Figures 3 and 4 present the F1-score violin plots and box plots of 16 algorithms on nine representative datasets. The box plot shows the median, interquartile range, whiskers and outliers of the results obtained from 30 independent runs. A higher box position indicates a better F1-score, while a shorter vertical span suggests a more concentrated distribution and stronger stability. The violin plot further shows the density of the performance distribution, where a wider region indicates that more runs are concentrated around the corresponding F1-score value. The inner box and median marker provide complementary information on the central tendency and dispersion. Thus, the box plot is useful for observing the variability and outliers, whereas the violin plot helps determine whether the 30 runs are concentrated in high-performance regions. As shown in Figure 3, the FOSCA maintained relatively high box positions on DS1, DS2, DS3, DS4 and DS7, together with comparatively narrow interquartile ranges. The high, narrow box distributions indicate that the FOSCA combined strong F1-scores with a stable performance across independent runs. In particular, on DS2, DS3 and DS4, the median F1-score of the FOSCA was clearly among the top-performing algorithms, while its fluctuation range remained relatively small, demonstrating strong reliability on these datasets.

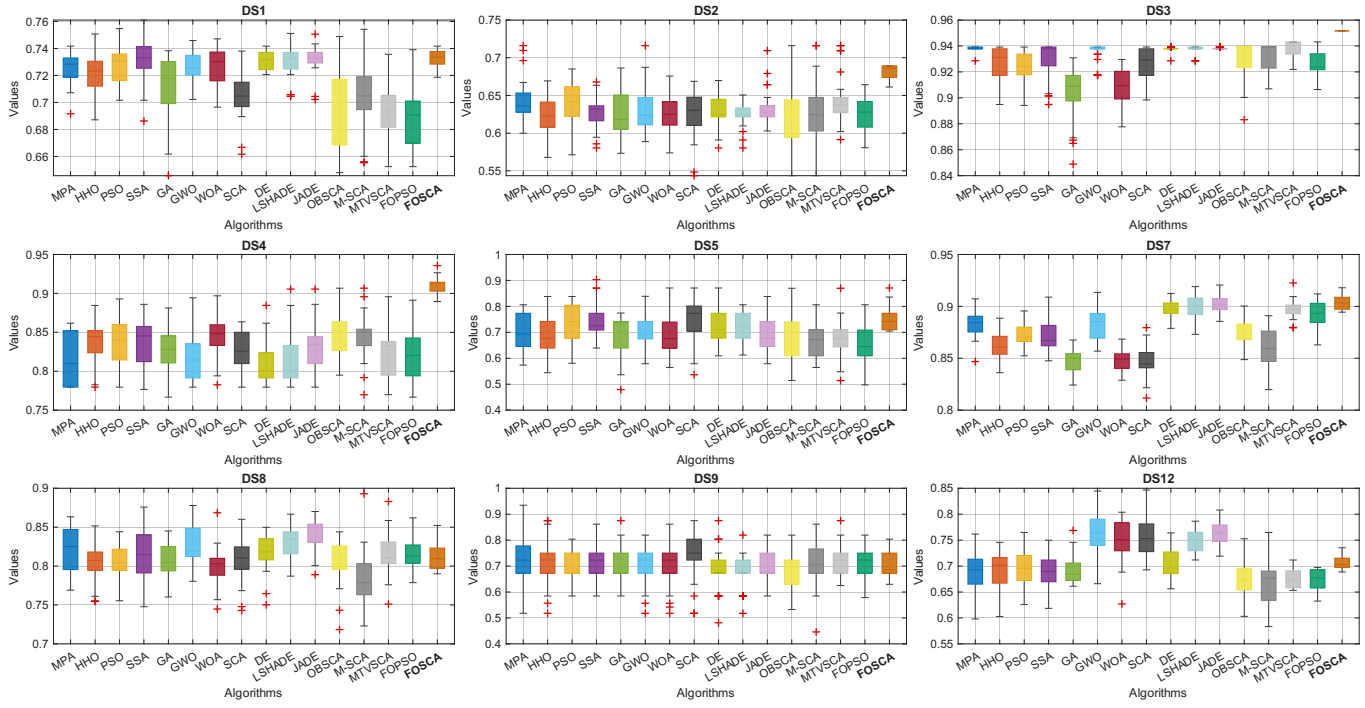

Figure 3. Box plots of FOSCA and other algorithms (F1-score).

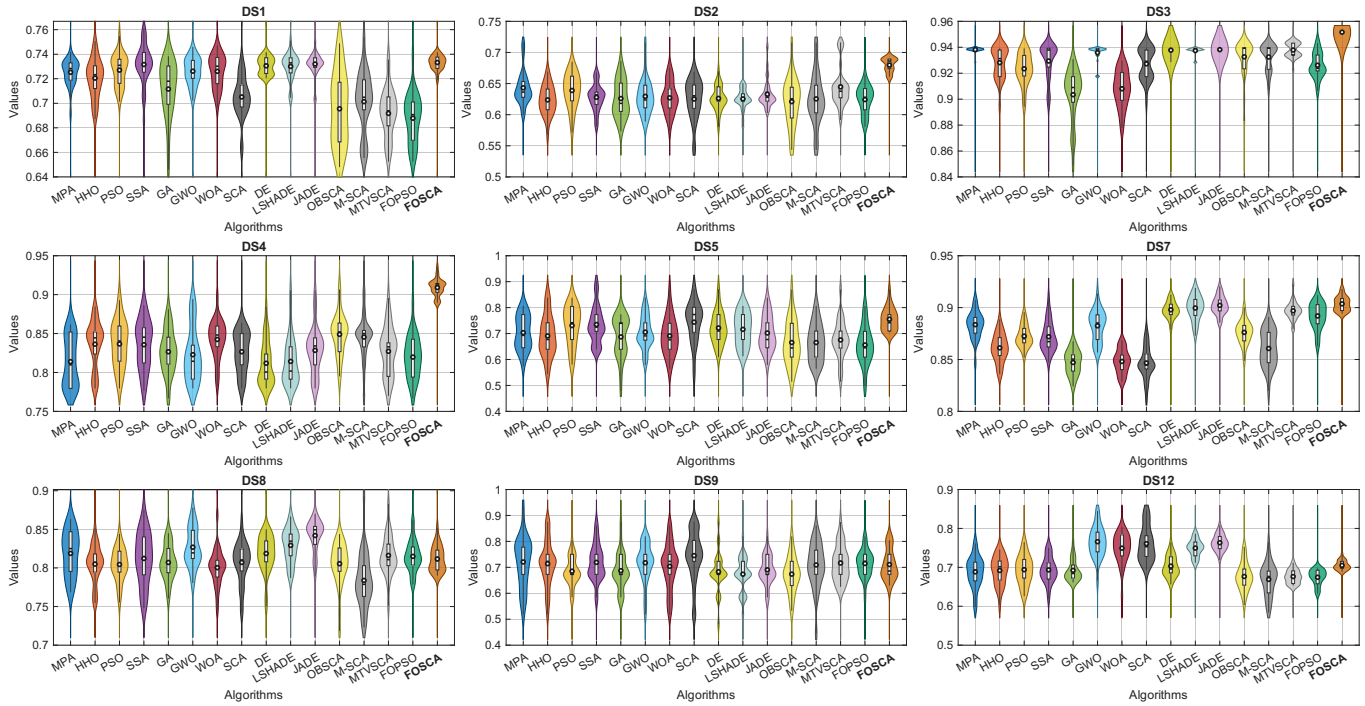

Figure 4. Violin plots of FOSCA and other algorithms (F1-score).

The violin distributions in Figure 4 further support the above observations. Compared with several competing algorithms, the FOSCA showed more concentrated distribution patterns on multiple datasets, with its main density regions located in higher F1-score intervals. Concentration in the higher F1-score range suggests that the observed performance recurred across runs rather than arising from isolated successes. On DS5, DS8, DS9 and DS12, although the FOSCA did not always achieve the highest median value, its distribution did not exhibit excessive dispersion and its overall performance remained highly competitive. Fractional-order reconstruction, elite-center guidance, and greedy acceptance may have jointly contributed to the observed stability. Therefore, the box plots and violin plots jointly demonstrate that the FOSCA is not only competitive in terms of the average F1-score, but also superior or comparable to most competing algorithms in terms of stability and reliability across repeated independent runs.

4.4. Convergence Behavior Analysis of FOSCA

Figure 5 compares the mean best-so-far fitness of the FOSCA and 11 competing algorithms on nine datasets under 10,000 function evaluations. The FOSCA reduced the fitness rapidly during the early stage on all datasets. It continued to improve throughout the search. Late-stage improvements were most evident on DS7-DS9. The FOSCA also maintained low fitness trajectories on DS3, DS8, and DS9. Its final position varied on the other datasets. Overall, the curves indicate a shift from early exploration to stronger exploitation.

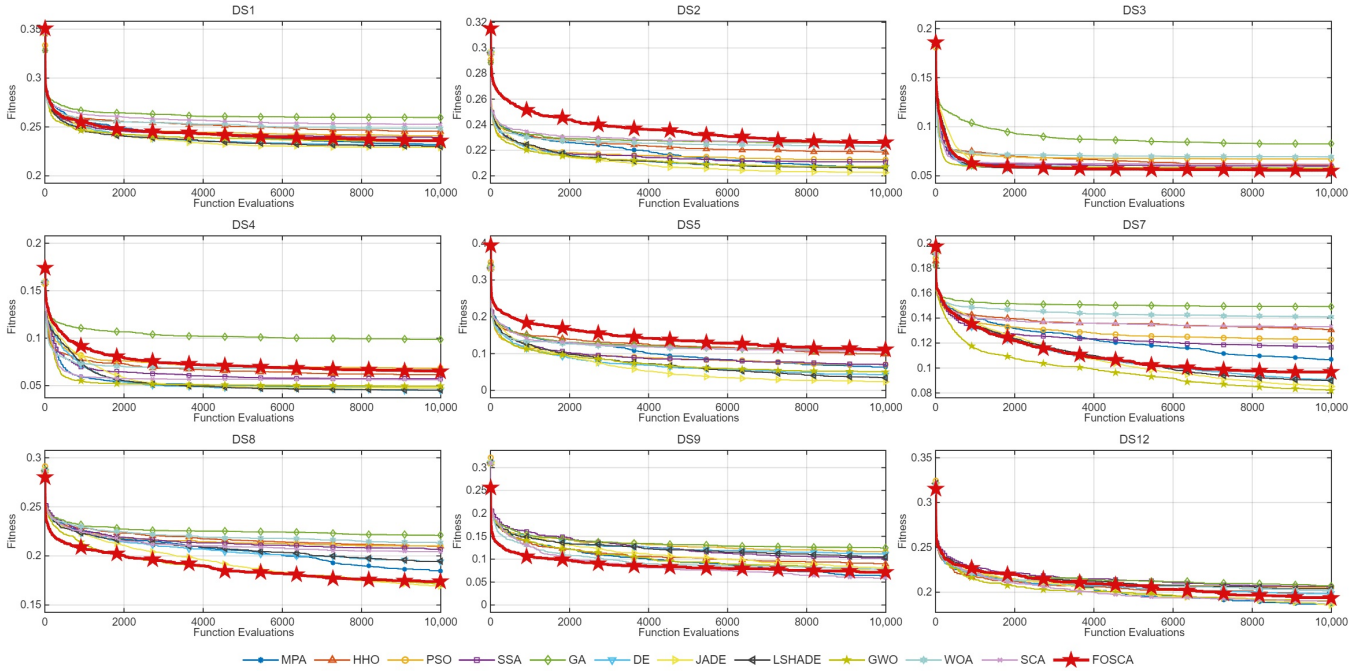

Figure 5. Convergence curves of FOSCA and the comparative algorithms.

4.5. Statistical Tests

To assess whether the performance differences among the algorithms were statistically significant, this study employed the Friedman ranking test and the Wilcoxon signed-rank test.

All tests used a 95% confidence level ($\alpha = 0.05$). For each metric mean obtained from 30 independent runs, the two-sided 95% t-based confidence interval was estimated as

$$\mathrm{CI}_{95\%} = \bar{x} \pm t_{0.975,29} \frac{s}{\sqrt{30}}, \tag{34}$$

where $\bar{x}$ and s denote the sample mean and standard deviation. A Holm-adjusted $p < 0.05$ indicated a significant pairwise difference.

4.5.1. Friedman Overall Ranking Analysis

The Friedman test was used to rank the algorithms based on four classification metrics: the accuracy, F1-score, precision and recall. The results are presented in Figure 6. In the lollipop chart, each point represents the average Friedman rank of the corresponding algorithm over the 14 datasets. A smaller value on the horizontal axis indicates a better overall ranking; therefore, algorithms closer to the left side showed a stronger comprehensive performance. The FOSCA achieved the smallest average rank on all four classification metrics, with ranks of 3.857, 4.214, 3.857 and 3.571 for the accuracy, F1-score, precision and recall, respectively. The FOSCA ranked better than every comparator across all four classification metrics. The Friedman test results for all four metrics also reached the significance level, with p-values of (9.823×10^{-5}), (4.375×10^{-5}), (1.805×10^{-5}) and (2.730×10^{-5}) for the accuracy, F1-score, precision and recall, respectively. The Friedman p-values indicate significant between-algorithm differences across all four metrics. The best average rank of the FOSCA across all classification metrics further confirms its overall advantage in feature-selection tasks.

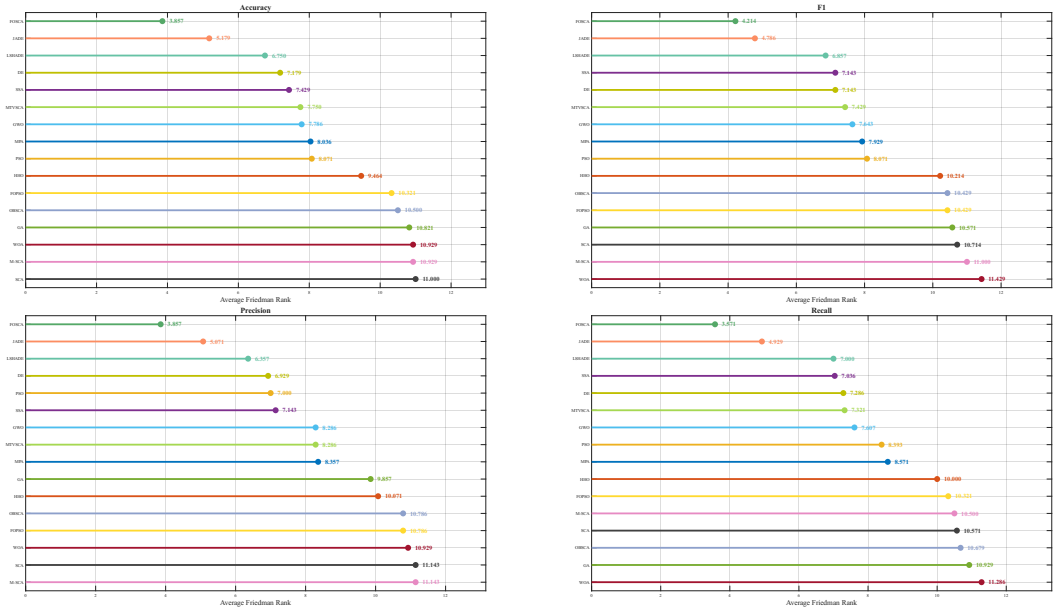

Figure 6. Firedman ranking of FOSCA compared to other algorithms (lollipop chart). Accuracy: Friedman $\chi^2 = 44.313$, $p = 9.823 \times 10^{-5}$. F1-score: Friedman $\chi^2 = 46.531$, $p = 4.375 \times 10^{-5}$. Precision: Friedman $\chi^2 = 48.920$, $p = 1.805 \times 10^{-5}$. Recall: Friedman $\chi^2 = 47.809$, $p = 2.730 \times 10^{-5}$. A p-value < 0.05 indicates statistically significant differences among the compared algorithms, while the Friedman χ^2 quantifies the overall extent of the ranking differences.

Figure 7 further illustrates the radar ranking results of 16 algorithms on 14 datasets across six indicators. In addition to the classification metrics, we included the NumFeat to provide a more comprehensive analysis from the perspectives of the optimization quality and feature subset compactness. In the radar chart, each subplot corresponds to one algorithm and the colored polylines represent the ranking results of the fitness, accuracy, recall, precision, F1-score and NumFeat across different datasets. Since a smaller rank value indicates a better performance, a polyline closer to the center reflects a stronger ranking performance on the corresponding dataset and metric. The value in parentheses denotes the overall average rank of each algorithm across all datasets and indicators. As shown in Figure 7, the FOSCA achieved an overall average rank of 4.250, ranking first among all algorithms and outperforming strong competitors such as the JADE, GWO, MPA, LSHADE and DE. More importantly, the FOSCA maintained favorable rankings not only on classification metrics such as the accuracy, recall, precision and F1-score, but also on NumFeat. This indicates that its performance gain is not achieved simply by selecting more features. Instead, the FOSCA strikes a more effective balance between the classification performance and feature subset compactness.

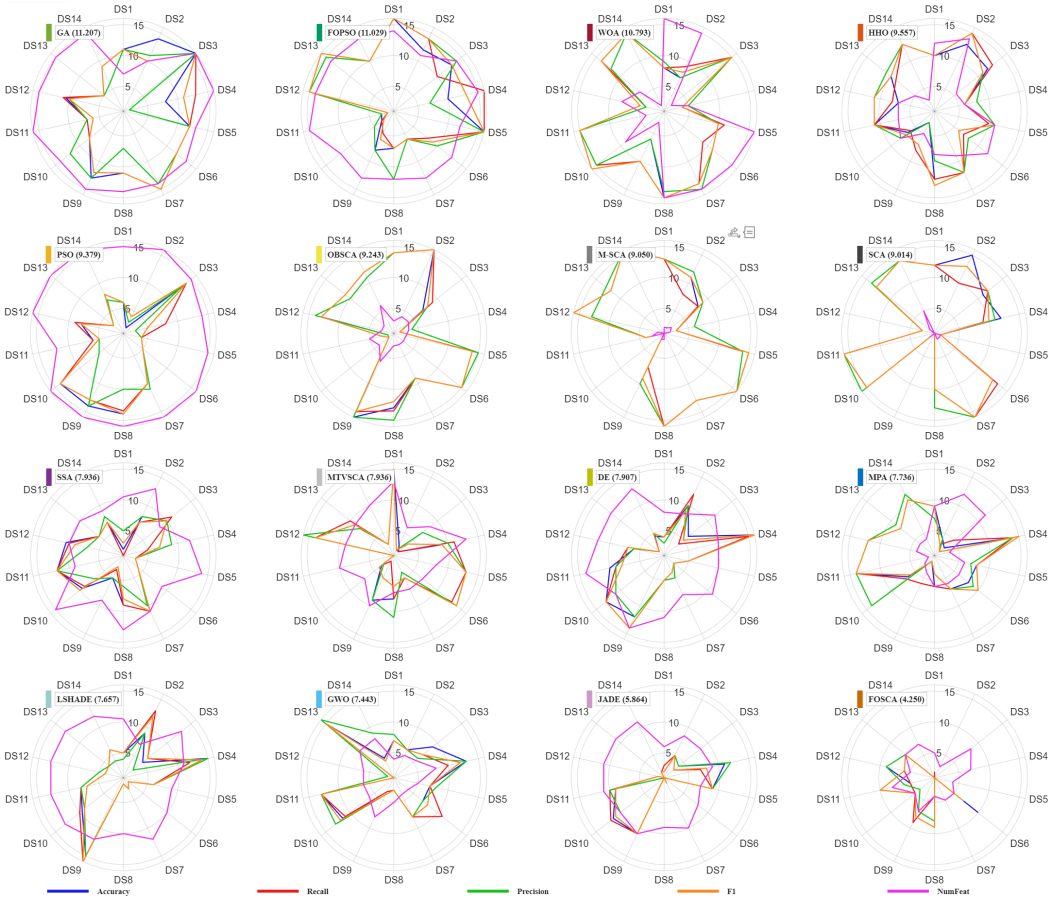

Figure 7. Firedman ranking radar charts for FOSCA and other algorithms on each dataset.

4.5.2. Wilcoxon Signed-Rank Test

To further investigate the pairwise differences between the FOSCA and each competing algorithm, the Wilcoxon signed-rank test was performed on the accuracy, F1-score, precision and recall over 14 datasets. The Holm correction was further applied to control the significance error caused by multiple comparisons. Figure 8 presents the significance comparison results of the FOSCA against the compared algorithms. In each subplot, the horizontal axis denotes the competing algorithms and the vertical axis denotes the datasets. The symbols "+", "≈" and "−" indicate that the FOSCA is significantly better than, statistically comparable to and significantly worse than the corresponding algorithm, respectively. Green "+" cells dominate all four metric panels, indicating significant FOSCA advantages in most algorithm–dataset comparisons. In particular, on DS2, DS3, DS4, DS5, DS7, DS10 and DS14, the FOSCA significantly outperformed or remained statistically comparable to most algorithms. The breadth of significant comparisons argues against gains driven by only a few successful runs. In contrast, the "−" symbols mainly appear on a limited number of datasets and against several strong competitors, such as the JADE, LSHADE, DE, or some SCA variants. The remaining losses suggest that search mechanisms retain dataset-dependent strengths under particular data distributions.

Figure 9 further summarizes the win/tie/loss results of the FOSCA against each competing algorithm across all datasets and metrics. The stacked bar chart on the left reports 56 pairwise comparisons formed by 14 datasets and four metrics, where green, blue and pink denote the wins, ties and losses of the FOSCA, respectively. The heatmap on the right provides the detailed numbers of wins, ties and losses for each metric. Overall, the FOSCA obtained positive comprehensive scores against all competing algorithms. Its scores against the M-SCA, SCA, MPA, WOA and OBSCA reached 0.71, 0.64, 0.64, 0.62 and

0.61, respectively. Pairwise comparisons favored the FOSCA over both conventional swarm optimizers and several improved SCA variants. Taking the M-SCA and SCA as examples, the FOSCA achieved win/tie/loss results of 44/8/4 and 40/12/4 out of 56 comparisons, respectively. This demonstrates that the proposed fractional-order memory, dynamic elite-center guidance and fast-decaying search factor can effectively improve the search stability and classification performance of the original SCA and its variants in feature-selection tasks. In summary, the Wilcoxon–Holm statistical analysis further verified the significant advantage and comprehensive robustness of the FOSCA under multi-dataset and multi-metric conditions.

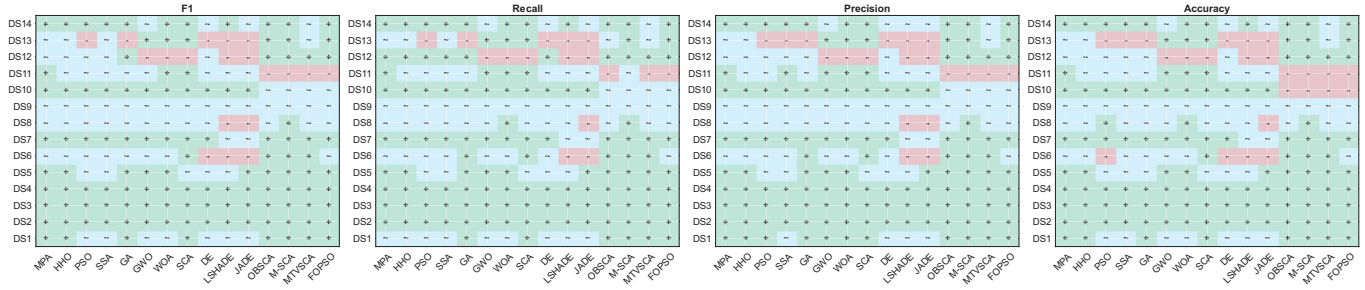

Figure 8. FOSCA pairwise Wilcoxon outcome symbols across datasets.

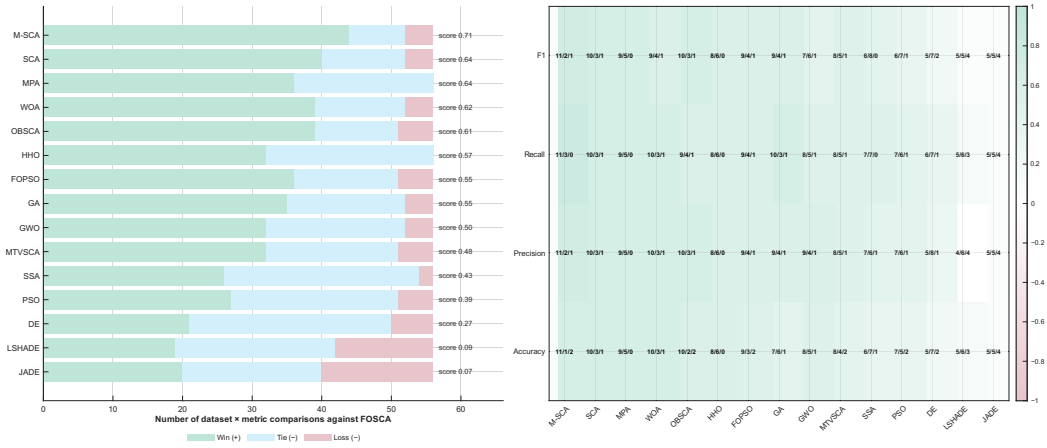

Figure 9. FOSCA W/T/L summary against comparative algorithms. The formula for calculating the score in the left image is $WTLscore = (Win - Loss)/Total$.

4.6. Parameter Sensitivity Analysis

The fractional order α affects both the short-memory position reconstruction and the nonlinear decay of the search factor in the FOSCA. To evaluate this coupled effect, α was varied over $\{0.25, 0.50, 0.75, 0.82, 0.95\}$, while all other parameters and the function-evaluation budget were kept unchanged. DS1, DS9, and DS14 were selected to cover feature spaces of markedly different sizes, containing 18, 2000, and 7129 features, respectively. The experimental results are shown in Figures 10 and 11.

The influence of α can first be characterized analytically. The fractional memory position is identical to the definition in Equations (13) and (14):

$$\mathbf{x}^{t}_{\mathrm{mem},i} = c_0\mathbf{x}^{t}_{i} + c_1\mathbf{x}^{t-1}_{i} + c_2\mathbf{x}^{t-2}_{i}, \tag{35}$$

where

$$c_0 = \alpha, \quad c_1 = \frac{1}{2}\alpha(1-\alpha), \quad c_2 = \frac{1}{6}\alpha(1-\alpha)(2-\alpha), \quad 0 < \alpha < 1. \tag{36}$$

Although c_1 and c_2 reach their absolute maxima at $\alpha = 0.5$ and $\alpha = 1 - \sqrt{3}/3$, respectively, their contributions relative to the current state,

$$\frac{c_1}{c_0} = \frac{1-\alpha}{2}, \qquad \frac{c_2}{c_0} = \frac{(1-\alpha)(2-\alpha)}{6}, \tag{37}$$

decrease monotonically with α. A larger α therefore places greater relative emphasis on the current position and weakens historical correction. At the default setting $\alpha = 0.82$, $(c_0, c_1, c_2) = (0.8200, 0.0738, 0.0290)$, yielding a current-state-dominated update with a small short-term memory contribution.

From Equations (15) and (22), the same parameter also controls the nonlinear search factor.

$$r_1(\tau, \alpha) = 2(1-\tau)^{1/\alpha}, \qquad \tau = \frac{\mathrm{FE}}{\mathrm{FE}_{\max}}. \tag{38}$$

For $0 < \tau < 1$,

$$\frac{\partial r_1}{\partial \alpha} = -\frac{2(1-\tau)^{1/\alpha} \ln(1-\tau)}{\alpha^2} > 0, \tag{39}$$

and the cumulative perturbation budget is

$$B(\alpha) = \int_0^1 r_1(\tau, \alpha)\, \mathrm{d}\tau = \frac{2\alpha}{1+\alpha}, \qquad \frac{\mathrm{d}B}{\mathrm{d}\alpha} > 0. \tag{40}$$

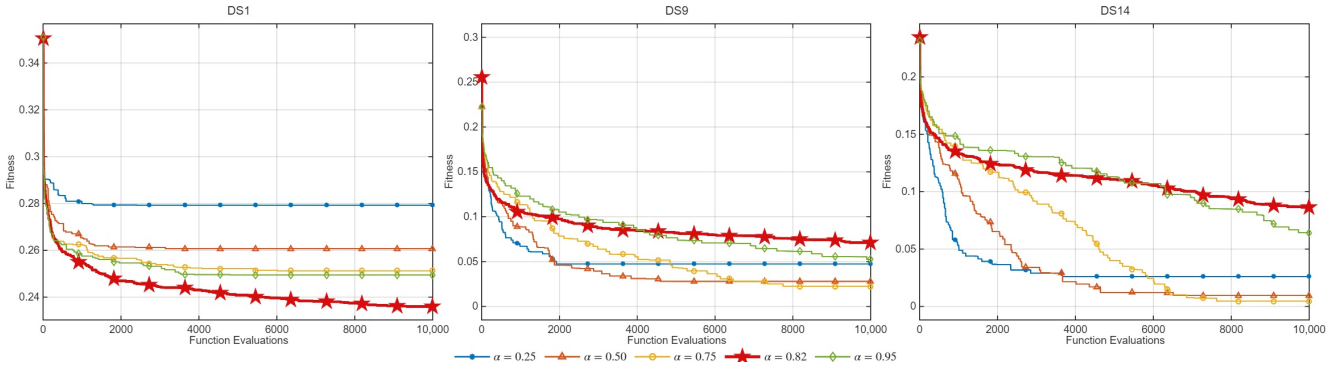

Figure 10. Convergence behavior of FOSCA under different fractional orders α.

Thus, increasing α weakens the relative historical dependence while preserving a larger search amplitude, whereas decreasing it strengthens historical correction, but accelerates contraction. Figures 10 and 11 show that the preferred setting was dataset-dependent. The lowest final fitness occurred at $\alpha = 0.82$ on DS1 and at $\alpha = 0.75$ on DS9 and DS14. Nevertheless, $\alpha = 0.82$ attained the highest recall, precision, and F1-score on all three datasets, together with the highest accuracy on DS1 and DS14. Across the tested settings, $\alpha = 0.82$ provided a robust default for the classification metrics, but was not universally optimal for the fitness or sparsity.

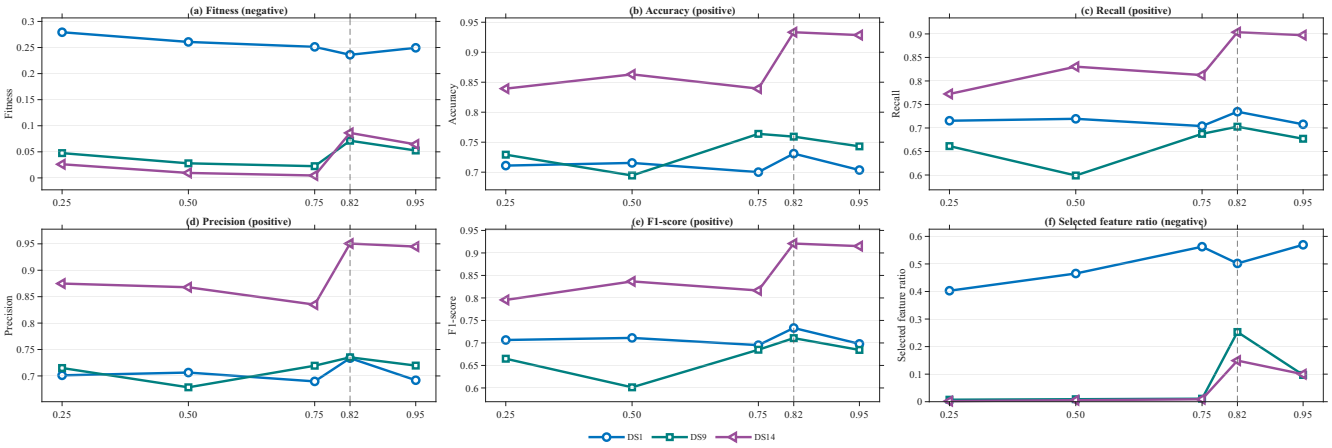

Figure 11. Sensitivity of the FOSCA to the fractional order α across the fitness, classification performance, and selected feature ratio. The fitness and selected feature ratio are minimized, whereas the accuracy, recall, precision, and F1-score are maximized. The dashed line marks the default setting $\alpha = 0.82$.

4.7. Effectiveness of the Involved Strategies

This paper designed ablation experiments to verify the effectiveness of the proposed components in the FOSCA and compare different variants with the original SCA as a baseline. The details of these variants are described as follows:

- FOSCAV1 removes the dynamic elite-center guidance mechanism while retaining the alpha-coupled fast (r_1) schedule and the three-term GL fractional memory. This variant was used to examine whether replacing the single global optimal guidance in the SCA with a dynamic elite-center guide contributes to the final performance.
- FOSCAV2 removes the alpha-coupled fast (r_1) schedule and restores the standard linear (r_1) decay of the SCA, while keeping the elite-center guidance and the GL fractional memory. This variant evaluates whether the proposed nonlinear exploration-to-exploitation transition is beneficial for feature selection.
- FOSCAV3 removes the three-term Grünwald–Letnikov fractional memory and directly uses the current position in the update equation, while retaining the elite-center guidance and the fast (r_1) schedule. This variant was designed to verify the contribution of historical memory information to the search process.
- FOSCAV4 is a variant of truncated memory ablation. It approximates and simplifies the three fractional memories in the full FOSCA into two forms. This variant was developed to analyze the necessity of deeper fractional memories within the FOSCA framework. Equation (13) was modified to: $\mathbf{x}^t_{\mathrm{mem},i} = c_0\mathbf{x}^t_i + c_1\mathbf{x}^{t-1}_i$.

Figure 12 presents the F1-score gain heatmap of the ablation variants on the 14 datasets, where each cell represents ($\Delta F1 = F1_{\mathrm{variant}} - F1_{\mathrm{SCA}}$). The color transition from purple to cyan-green denotes a performance degradation or improvement relative to the SCA, respectively, while the bar chart on the right summarizes the average gain on each dataset. Most datasets and variants showed positive gains. Thus, the improvement was distributed across tasks rather than driven by an isolated advantage on one dataset. In particular, the complete FOSCA maintained positive gains on most datasets and recovered a favorable F1-score performance on several datasets where some incomplete variants degraded. Recovery by the complete model suggests complementary roles for the three mechanisms.

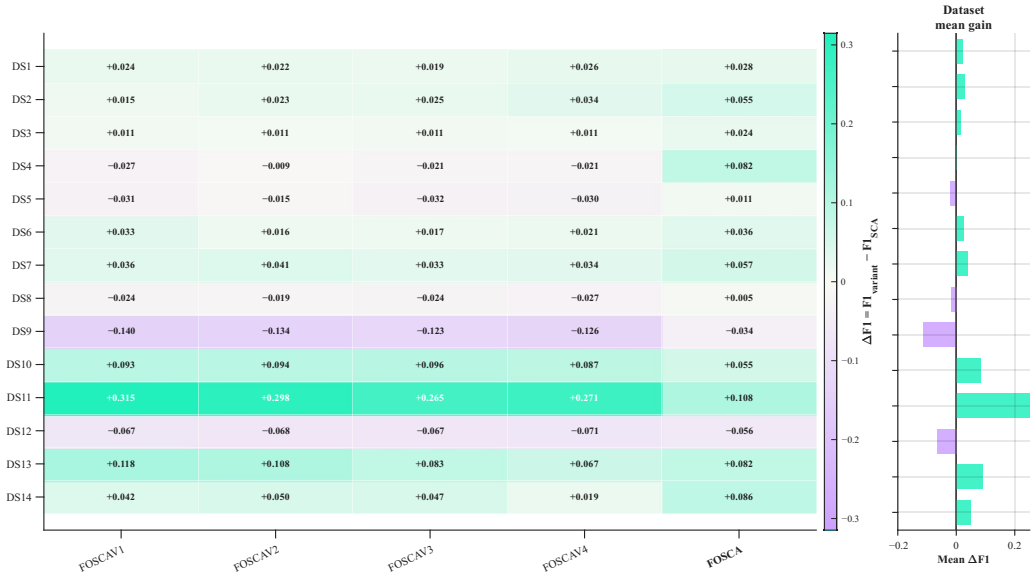

Figure 12. Ablation gain heatmap.

To quantify each component, we averaged ($\Delta F1 = F1_{\text{variant}} - F1_{\text{SCA}}$) across the 14 datasets. Dynamic elite-center guidance, nonlinear r_1 decay, and three-term GL memory increased the mean F1-score by 0.028, 0.030, and 0.024, respectively. The FOSCA achieved a higher mean F1-score than each corresponding ablation on 11 of the 14 datasets. Relative to FOSCAV4, the full three-term memory added 0.021 and achieved higher values on 12 datasets. Overall, the FOSCA improved the mean F1-score by 0.039 over the SCA and performed better on 12 datasets.

The complete FOSCA produced a more stable gain distribution than the ablated variants. This pattern suggests that the mechanisms act jointly rather than through one dominant module. Meanwhile, negative gains were still observed on a few datasets, suggesting that the effectiveness of the FOSCA may be partially dataset-dependent. Dataset-dependent losses motivate adaptive mechanism selection based on data characteristics.

4.8. Normalized Comprehensive Performance Analysis

A multi-metric normalized performance heatmap was constructed to evaluate the comprehensive competitiveness of different algorithms in feature selection from a global perspective. As shown in Figures 13 and 14, the heatmaps provide complementary views of normalized algorithm performance. Specifically, for Figure 13, the mean values of five metrics over 30 independent runs were first calculated for each dataset and algorithm. Min-max normalization was then performed according to the optimization direction of each metric. After normalization, all metrics were mapped into the interval $[0, 1]$, where a larger value indicates a better relative performance on the corresponding metric. Normalization makes the heatmap comparable across metrics with different scales and optimization directions.

From the algorithm–metric heatmap shown in Figure 14, the FOSCA achieved high normalized scores on classification-related metrics, including the accuracy, recall, precision and F1-score, indicating its stable advantage in predictive performance. Although some traditional algorithms performed better on NumFeat, suggesting that they can select fewer features, their scores on the main classification metrics were relatively lower. Their greater sparsity may therefore come at the cost of classification performance. In contrast, the FOSCA showed a more balanced and competitive performance across multiple classification metrics while maintaining reasonable feature-reduction capability. The dataset–algorithm heatmap places the FOSCA at or near the highest normalized F1-score on most datasets. Its advantage therefore extends beyond a single data distribution. Overall, the multi-metric

normalized heatmap further confirms that the FOSCA achieved a better balance among the classification performance, feature-selection quality and comprehensive robustness.

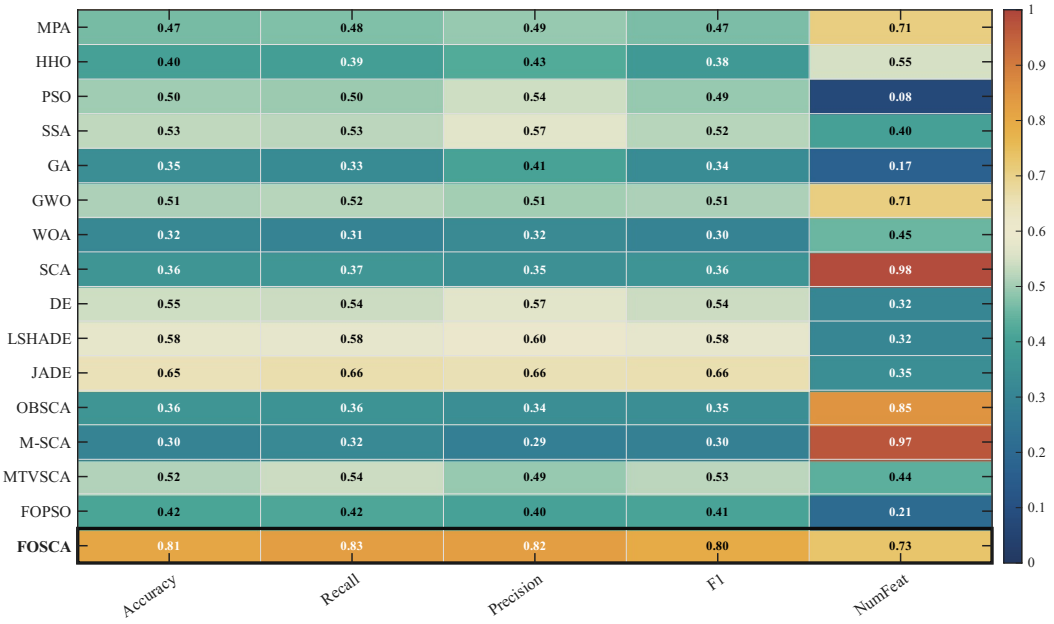

Figure 13. Algorithms × metrics—normalized heatmap.

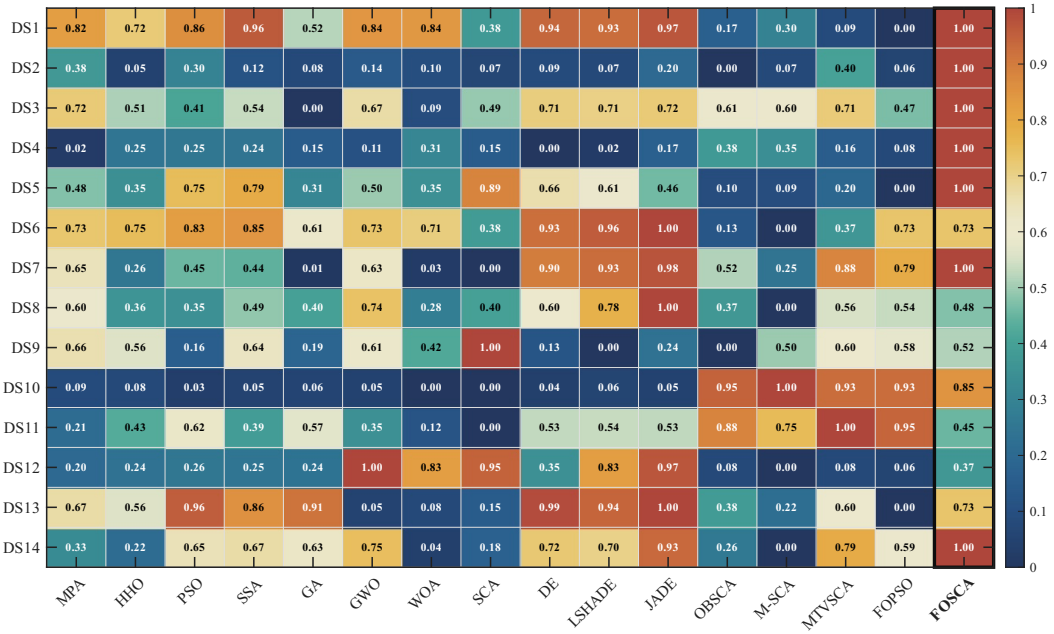

Figure 14. Datasets × algorithms—normalized heatmap.

4.9. Performance–Sparsity Trade-Off Analysis

To verify that the performance improvement of the FOSCA was not achieved by selecting more features, a performance–sparsity Pareto front analysis was conducted from a multi-objective trade-off perspective. Figure 15 plots the cross-dataset mean feature-selection ratio on the horizontal axis. For each dataset, this ratio divides the 30-run mean selected-feature count by the original dimensionality. The vertical axis represents the cross-dataset average values of the accuracy, F1-score, recall, and precision, respectively. Therefore, algorithms closer to the upper-left region are more desirable, as they achieved a better classification performance with fewer selected features.

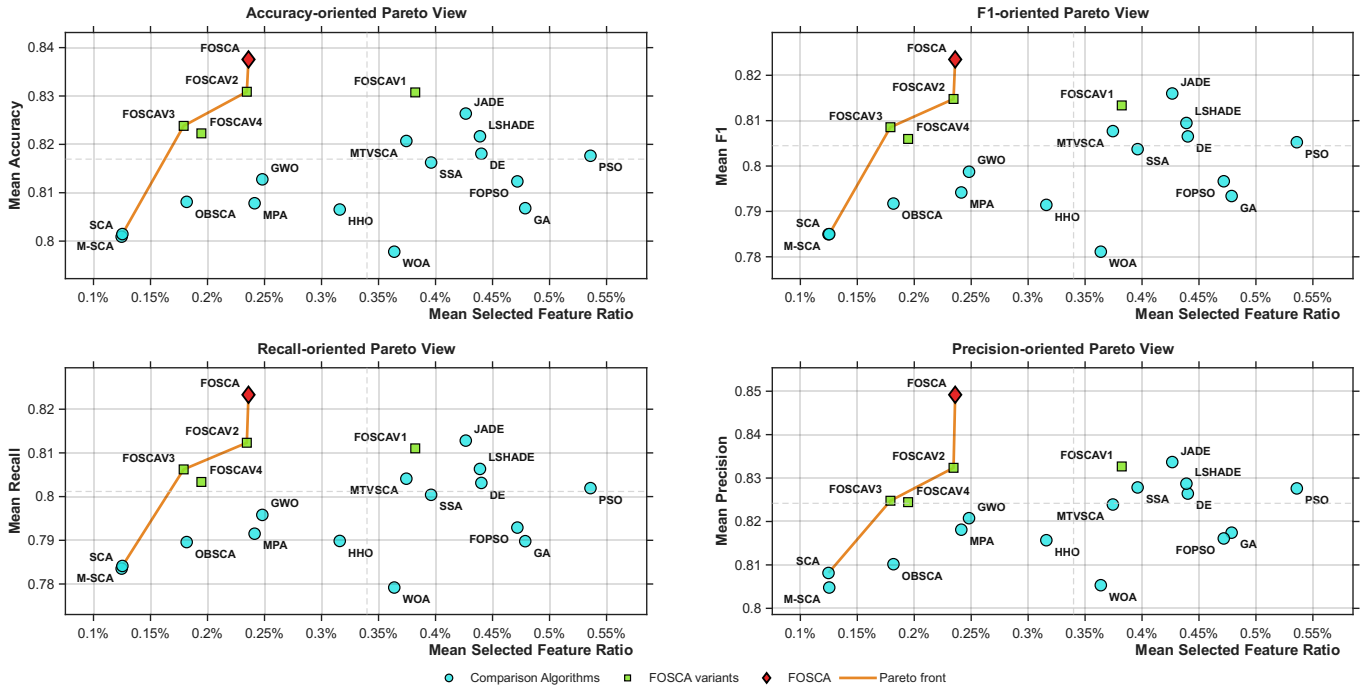

Figure 15. Performance–sparsity Pareto fronts across 14 datasets.

From the results of the four metrics, the FOSCA was consistently located in the high-performance region of the Pareto front in terms of its accuracy, F1-score, recall and precision. Moreover, its average feature-selection ratio was clearly lower than that of most strong competitors, such as the JADE, LSHADE, DE, PSO and FOPSO. The FOSCA's gain therefore cannot be explained by selecting a larger feature subset. Instead, it achieved a superior overall performance with a more compact set of features. In contrast, although the SCA and M-SCA obtained lower feature-selection ratios, their classification performance remained relatively insufficient. Meanwhile, several high-performing competing algorithms usually require more selected features to maintain their predictive accuracy. Compared with FOSCAV1-FOSCAV4, the FOSCA also shows a more stable advantage, suggesting that the complete integration of the proposed mechanisms further improves the balance between the classification performance and feature sparsity. Overall, the Pareto analysis supports a favorable performance–sparsity trade-off for the FOSCA and complements the classification results.

4.10. CPU Running-Time Consumption of Algorithms

Figures 16 and 17 compare the CPU runtimes of 20 algorithms, including four FOSCA ablation variants, across 14 datasets under an identical function-evaluation budget. Figure 16 reports the dataset-wise runtimes together with their relative and arithmetic means, whereas Figure 17 highlights the runtime variations across datasets.

As shown in Figure 16, the FOSCA achieved an average runtime of 379.5 s, ranking eighth with a relative runtime ratio of 1.53×. Although fractional-memory reconstruction and elite guidance introduced additional overhead relative to the SCA, the FOSCA remained faster than 11 of the 15 competing methods. The runtimes of FOSCAV2-FOSCAV4 further indicated that the α-coupled fast r_1 schedule and the full three-term Grünwald–Letnikov memory reconstruction accounted for part of this overhead. Moreover, as shown in Figure 17, the runtime peaks observed for most algorithms on DS7 and DS12 showed that the computational cost was strongly dataset-dependent. Despite its added search mechanisms, the FOSCA retained a competitive computational efficiency.

Figure 16. Datasets × time array runtime—heatmap average combined. Relative time uses the fastest algorithm within each dataset as 1.00×; the average runtime is the arithmetic mean of all 14 datasets. Rows are ranked by the arithmetic mean runtime from fastest to slowest; the FOSCA is highlighted in red.

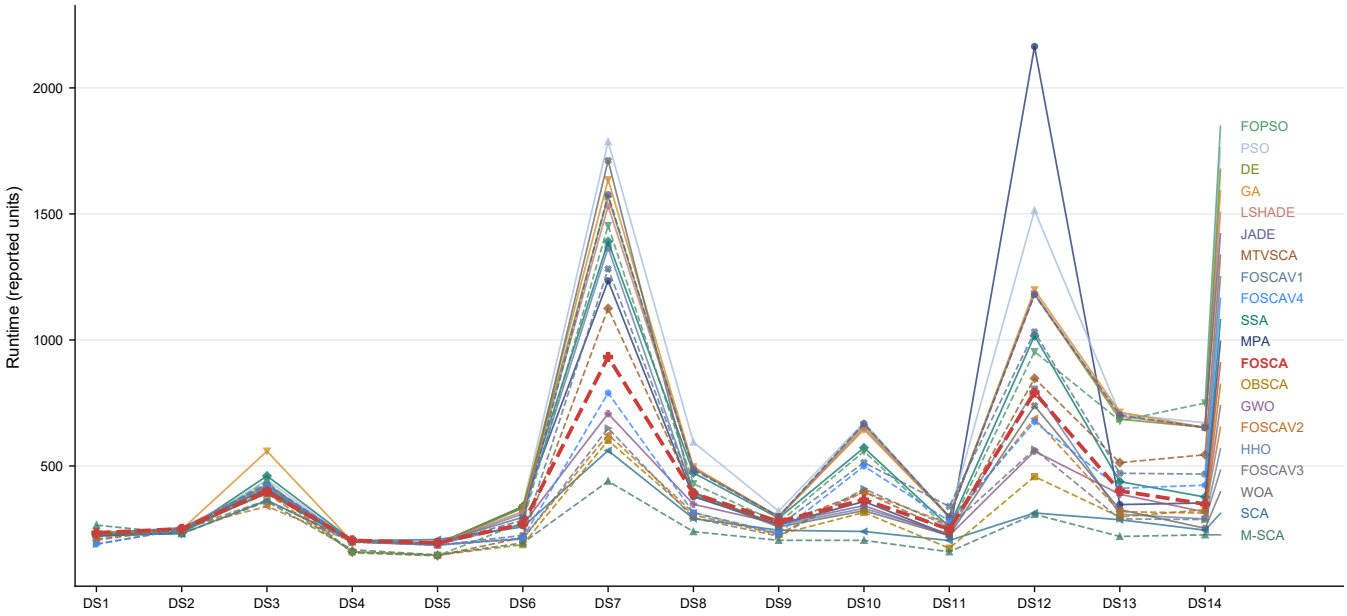

Figure 17. Computational runtime on all the datasets.

5. Discussion and Future Directions

The proposed FOSCA redesigns the continuous dynamic search of the original SCA framework, making it more suitable for discretized feature-selection tasks. The experimental results indicate that the short-memory fractional-order position reconstruction, dynamic elite-center guidance and α-coupled fast-decaying search factor work in a complementary manner. Specifically, the fractional-order memory helps exploit recent historical search information and smooth the search trajectory; the dynamic elite center alleviates the excessive dependence on a single global optimal; and the fast-decaying search factor reduces ineffective late-stage oscillations. Consequently, the FOSCA achieved strong competitiveness not only in classification-oriented metrics, including the accuracy, precision, recall and F1-score,

but also in the repeated-run stability, statistical significance and performance–sparsity trade-off. These observations suggest that improving the internal dynamic search of the SCA can effectively enhance its search reliability in wrapper-based feature selection, rather than merely obtaining performance gains through changes in the external classifier or evaluation protocol.

Nevertheless, several issues remain worthy of further investigation. First, the current FOSCA uses a fixed fractional order α and a predefined elite-center contraction strategy. Although this setting yielded a stable performance in the present experiments, datasets with different dimensional scales, class distributions and feature redundancy levels may require different search behaviors. Therefore, fixed parameters may not always match the most appropriate search state. Second, KNN was adopted as the unified wrapper classifier in this study to ensure a fair comparison, but different learning models may exhibit different sensitivities to the selected feature subsets. Thus, the generalization ability of the FOSCA under other classifiers, such as an SVM, random forests, extreme learning machines, or deep models, deserves further validation. In addition, the elite sorting and fractional-order memory mechanisms introduce internal search overhead. Although this overhead is usually not the dominant computational cost in wrapper-based feature selection, where repeated subset evaluation is more expensive, the computational efficiency should still be further optimized for large-scale data or more complex classifiers.

Future work can be extended in three directions:

- An adaptive fractional-order mechanism can be developed so that α, the memory length and the search amplitude are dynamically adjusted according to the population diversity, convergence speed, or fitness improvement rate, thereby enhancing the adaptability of the FOSCA to different data characteristics.
- The FOSCA can be extended into a multi-objective feature-selection framework that directly optimizes the classification error and the number of selected features simultaneously, instead of combining them into a single fitness function using a fixed weight. This extension may provide a more complete Pareto solution set for different application requirements.
- The stability and interpretability of the selected feature subsets can be further investigated, for example, by analyzing frequently selected features, the cross-run Jaccard similarity and the relationship between selected features and the class-discriminative ability.

Through these extensions, the FOSCA has the potential to evolve from a performance-oriented wrapper optimizer into a feature-selection framework with a high accuracy, a strong stability and improved interpretability.

Author Contributions: Conceptualization, S.G.; Methodology, S.G.; Software, Y.X.; Validation, Y.X. and W.L.; Formal Analysis, W.L.; Investigation, Y.X. and K.X.; Resources, Y.X. and W.L.; Data Curation, W.L. and K.X.; Writing—Original Draft, Y.X.; Writing—Review and Editing, S.G. and B.Q.; Visualization, Y.X.; Supervision, S.G. and B.Q.; Project Administration, S.G. and B.Q.; Funding Acquisition, S.G. and B.Q. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the National Natural Science Foundation of China under grant No. 62376109 and the National Undergraduate Innovation and Entrepreneurship Training Program of Jiangsu University of Science and Technology (project No. X2026102890014).

Data Availability Statement: The data used in this study must be authentic and reliable to ensure the validity and reproducibility of the experimental results. The datasets and source code used in this study are openly available at: https://github.com/xieeix-code/FOSCA (accessed on 24 June 2026).

Acknowledgments: The authors have reviewed and edited the output and take full responsibility for the content of this publication. During the preparation of this manuscript, the authors utilized ChatGPT 5.5 to assist with language refinement and stylistic improvement. All generated content was subsequently reviewed, verified and revised by the authors, who take full responsibility for the accuracy and integrity of the final publication.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Rostami, M.; Berahmand, K.; Nasiri, E.; Forouzandeh, S. Review of swarm intelligence-based feature selection methods. *Eng. Appl. Artif. Intell.* **2021**, *100*, 104210. [CrossRef]
2. Han, Q.; Hu, L.; Gao, W. Feature relevance and redundancy coefficients for multi-view multi-label feature selection. *Inf. Sci.* **2024**, *652*, 119747. [CrossRef]
3. Cicek, O.; Özçelik, Y.B.; Altan, A. A new approach based on metaheuristic optimization using chaotic functional connectivity matrices and fractal dimension analysis for AI-driven detection of orthodontic growth and development stage. *Fractal Fract.* **2025**, *9*, 148. [CrossRef]
4. Hu, Y.; Zhang, Y.; Gong, D. Multiobjective particle swarm optimization for feature selection with fuzzy cost. *IEEE Trans. Cybern.* **2020**, *51*, 874–888.
5. Wang, P.; Xue, B.; Liang, J.; Zhang, M. Differential evolution with duplication analysis for feature selection in classification. *IEEE Trans. Cybern.* **2022**, *53*, 6676–6689.
6. Nguyen, B.H.; Xue, B.; Zhang, M. A survey on swarm intelligence approaches to feature selection in data mining. *Swarm Evol. Comput.* **2020**, *54*, 100663. [CrossRef]
7. Wang, P.; Xue, B.; Liang, J.; Zhang, M. Multiobjective differential evolution for feature selection in classification. *IEEE Trans. Cybern.* **2021**, *53*, 4579–4593.
8. Song, X.F.; Zhang, Y.; Guo, Y.N.; Sun, X.Y.; Wang, Y.L. Variable-size cooperative coevolutionary particle swarm optimization for feature selection on high-dimensional data. *IEEE Trans. Evol. Comput.* **2020**, *24*, 882–895. [CrossRef]
9. Lemhadri, I.; Ruan, F.; Abraham, L.; Tibshirani, R. Lassonet: A neural network with feature sparsity. *J. Mach. Learn. Res.* **2021**, *22*, 1–29. [CrossRef]
10. Ma, X.A.; Liu, H.; Liu, Y.; Zhang, J.Z. Multi-label feature selection considering label importance-weighted relevance and label-dependency redundancy. *Eur. J. Oper. Res.* **2025**, *322*, 215–236. [CrossRef]
11. Cherepanova, V.; Levin, R.; Somepalli, G.; Geiping, J.; Bruss, C.B.; Wilson, A.G.; Goldstein, T.; Goldblum, M. A performance-driven benchmark for feature selection in tabular deep learning. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 41956–41979. [CrossRef]
12. Li, X.; Wang, Y.; Ruiz, R. A survey on sparse learning models for feature selection. *IEEE Trans. Cybern.* **2020**, *52*, 1642–1660. [CrossRef]
13. Li, Z.; Nie, F.; Wu, D.; Wang, Z.; Li, X. Sparse trace ratio LDA for supervised feature selection. *IEEE Trans. Cybern.* **2023**, *54*, 2420–2433.
14. Wang, P.; Xue, B.; Liang, J.; Zhang, M. Feature selection using diversity-based multi-objective binary differential evolution. *Inf. Sci.* **2023**, *626*, 586–606. [CrossRef]
15. Hijazi, N.; Aloqaily, M.; Ouni, B.; Karray, F.; Debbah, M. Harris hawks feature selection in distributed machine learning for secure iot environments. In *Proceedings of the ICC 2023-IEEE International Conference on Communications, Rome, Italy, 28 May–1 June 2023*; IEEE: New York, NY, USA, 2023; pp. 3169–3174.
16. Fathi, I.S.; El-Saeed, A.R.; Hassan, G.; Aly, M. Fractional Chebyshev Transformation for Improved Binarization in the Energy Valley Optimizer for Feature Selection. *Fractal Fract.* **2025**, *9*, 521. [CrossRef]
17. Jiao, R.; Nguyen, B.H.; Xue, B.; Zhang, M. A survey on evolutionary multiobjective feature selection in classification: Approaches, applications, and challenges. *IEEE Trans. Evol. Comput.* **2023**, *28*, 1156–1176.
18. Song, X.F.; Zhang, Y.; Gong, D.W.; Gao, X.Z. A fast hybrid feature selection based on correlation-guided clustering and particle swarm optimization for high-dimensional data. *IEEE Trans. Cybern.* **2021**, *52*, 9573–9586.
19. Zhao, H.; Tang, L.; Xie, L.G.; Li, J.Y.; Liu, J. KMoPSO TE: Advancing Self-Driving Networks with a Knowledge-Driven Multi-Objective Particle Swarm Optimization Algorithm for Traffic Engineering. *IEEE Trans. Evol. Comput.* **2026**. [CrossRef]
20. Wang, P.; Xue, B.; Liang, J.; Zhang, M. Feature clustering-assisted feature selection with differential evolution. *Pattern Recognit.* **2023**, *140*, 109523. [CrossRef]
21. Wang, Y.; Ran, S.; Wang, G.G. Role-oriented binary grey wolf optimizer using foraging-following and Lévy flight for feature selection. *Appl. Math. Model.* **2024**, *126*, 310–326. [CrossRef]
22. Sun, L.; Si, S.; Ding, W.; Wang, X.; Xu, J. TFSFB: Two-stage feature selection via fusing fuzzy multi-neighborhood rough set with binary whale optimization for imbalanced data. *Inf. Fusion* **2023**, *95*, 91–108. [CrossRef]

23. Peng, L.; Cai, Z.; Heidari, A.A.; Zhang, L.; Chen, H. Hierarchical Harris hawks optimizer for feature selection. *J. Adv. Res.* **2023**, *53*, 261–278. [CrossRef] [PubMed]
24. Nadimi-Shahraki, M.H.; Taghian, S.; Javaheri, D.; Sadiq, A.S.; Khodadadi, N.; Mirjalili, S. MTV-SCA: Multi-trial vector-based sine cosine algorithm. *Clust. Comput.* **2024**, *27*, 13471–13515. [CrossRef]
25. Gupta, S.; Deep, K.; Mirjalili, S.; Kim, J.H. A modified sine cosine algorithm with novel transition parameter and mutation operator for global optimization. *Expert Syst. Appl.* **2020**, *154*, 113395. [CrossRef]
26. Abd Elaziz, M.; Oliva, D.; Xiong, S. An improved opposition-based sine cosine algorithm for global optimization. *Expert Syst. Appl.* **2017**, *90*, 484–500. [CrossRef]
27. Özçelik, Y.B.; Altan, A. Overcoming nonlinear dynamics in diabetic retinopathy classification: A robust AI-based model with chaotic swarm intelligence optimization and recurrent long short-term memory. *Fractal Fract.* **2023**, *7*, 598. [CrossRef]
28. Emary, E.; Zawbaa, H.M.; Hassanien, A.E. Binary grey wolf optimization approaches for feature selection. *Neurocomputing* **2016**, *172*, 371–381. [CrossRef]
29. Mafarja, M.; Mirjalili, S. Whale optimization approaches for wrapper feature selection. *Appl. Soft Comput.* **2018**, *62*, 441–453. [CrossRef]
30. Taghian, S.; Nadimi-Shahraki, M.H. Binary sine cosine algorithms for feature selection from medical data. *arXiv* **2019**, arXiv:1911.07805.
31. Solteiro Pires, E.; Tenreiro Machado, J.; de Moura Oliveira, P.; Boaventura Cunha, J.; Mendes, L. Particle swarm optimization with fractional-order velocity. *Nonlinear Dyn.* **2010**, *61*, 295–301. [CrossRef]
32. Guo, W.; Wang, Y.; Zhao, F.; Dai, F. Riesz fractional derivative elite-guided sine cosine algorithm. *Appl. Soft Comput.* **2019**, *81*, 105481. [CrossRef]
33. Altarabichi, M.G.; Nowaczyk, S.; Pashami, S.; Sheikholharam Mashhadi, P. Fast Genetic Algorithm for feature selection—A qualitative approximation approach. In Proceedings of the Companion Conference on Genetic and Evolutionary Computation, Lisbon, Portugal, 15–19 July 2023; pp. 11–12.
34. Dokeroglu, T.; Deniz, A.; Kiziloz, H.E. A comprehensive survey on recent metaheuristics for feature selection. *Neurocomputing* **2022**, *494*, 269–296. [CrossRef]
35. Jiao, R.; Xue, B.; Zhang, M. A tri-objective method for bi-objective feature selection in classification. *Evol. Comput.* **2024**, *32*, 217–248. [CrossRef] [PubMed]
36. Mafarja, M.; Aljarah, I.; Heidari, A.A.; Faris, H.; Fournier-Viger, P.; Li, X.; Mirjalili, S. Binary dragonfly optimization for feature selection using time-varying transfer functions. *Knowl.-Based Syst.* **2018**, *161*, 185–204. [CrossRef]
37. Mirjalili, S. SCA: A sine cosine algorithm for solving optimization problems. *Knowl.-Based Syst.* **2016**, *96*, 120–133. [CrossRef]
38. Hu, H.; Shan, W.; Tang, Y.; Heidari, A.A.; Chen, H.; Liu, H.; Wang, M.; Escorcia-Gutierrez, J.; Mansour, R.F.; Chen, J. Horizontal and vertical crossover of sine cosine algorithm with quick moves for optimization and feature selection. *J. Comput. Des. Eng.* **2022**, *9*, 2524–2555. [CrossRef]
39. Xie, Y.; Li, W.; Zhong, C.; Gao, S.; Xu, K.; Tu, J.; Qin, B. Self-Adaptive AdamW-Guided Optimization: A Learning-Driven Metaheuristic for Solving Complex Real-World Engineering Problems. *Entropy* **2026**, *28*, 660. [CrossRef] [PubMed]
40. Gupta, S.; Deep, K.; Engelbrecht, A.P. A memory guided sine cosine algorithm for global optimization. *Eng. Appl. Artif. Intell.* **2020**, *93*, 103718. [CrossRef]
41. Deng, L.; Liu, S. A sine cosine algorithm guided by elite pool strategy for global optimization. *Appl. Soft Comput.* **2024**, *164*, 111946. [CrossRef]
42. Nasser, A.B.; Ghanem, W.A.H.; Saad, A.M.H.; Abdul-Qawy, A.S.H.; Ghaleb, S.A.; Alduais, N.A.M.; Din, F.; Ghetas, M. Depth linear discrimination-oriented feature selection method based on adaptive sine cosine algorithm for software defect prediction. *Expert Syst. Appl.* **2024**, *253*, 124266. [CrossRef]
43. Yu, Z.; Dong, H.; Sun, J.; Zhao, B. A Cooperative Co-evolutionary Salp Swarm Feature Selection Algorithm with Reverse Escape Strategy for Classification. In *Proceedings of the 2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS), Ocean Flower Island, China, 17–21 December 2023*; IEEE: New York, NY, USA, 2023; pp. 68–75.
44. Askari, Q.; Younas, I.; Saeed, M. Critical evaluation of sine cosine algorithm and a few recommendations. In Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion, Cancún, Mexico, 8–12 July 2020; pp. 319–320.
45. Liu, J.; Zhao, F.; Li, Y.; Zhou, H. A new global sine cosine algorithm for solving economic emission dispatch problem. *Inf. Sci.* **2023**, *648*, 119569. [CrossRef]
46. Abed-Alguni, B.H.; Alawad, N.A.; Al-Betar, M.A.; Paul, D. Opposition-based sine cosine optimizer utilizing refraction learning and variable neighborhood search for feature selection. *Appl. Intell.* **2023**, *53*, 13224–13260. [CrossRef] [PubMed]
47. Atici, F.M.; Chang, S.; Jonnalagadda, J. Grünwald–Letnikov Fractional Operators: From Past to Present. *Fract. Differ. Calc.* **2021**, *11*, 147–159. [CrossRef]
48. Cao, Y.; Xu, Y.; Du, X.; Zhong, R.; Yu, J.; Munetomo, M. Tri-subpopulation sigmoid-enhanced sine-cosine algorithm and its application to gene function prediction problem. *J. Supercomput.* **2025**, *81*, 1579.

49. Shinde, V.; Jha, R.; Mishra, D.K. Improved chaotic sine cosine algorithm (ICSCA) for global optima. *Int. J. Inf. Technol.* **2024**, *16*, 245–260. [CrossRef]
50. Li, Y.; Zhao, Y.; Liu, J. A levy flight sine cosine algorithm for global optimization problems. *Int. J. Distrib. Syst. Technol. (IJDST)* **2021**, *12*, 49–66. [CrossRef]
51. Ishtaiwi, A.; Al-Shamayleh, A.S.; Fakhouri, H.N. A Hybrid JADE–Sine Cosine Approach for Advanced Metaheuristic Optimization. *Appl. Sci.* **2024**, *14*, 10248. [CrossRef]
52. Liu, J.; Bi, C.; Chen, H.; Heidari, A.A.; Chen, H. Triangular-based sine cosine algorithm for global search and feature selection. *Sci. Rep.* **2025**, *15*, 12992. [CrossRef] [PubMed]
53. Zhang, X.; Tang, L.; Hou, S.; Gui, W. Secretary bird optimization algorithm incorporating independent thinking mechanism and sine-square step length for feature selection. *Sci. Rep.* **2026**, *16*, 4067. [CrossRef] [PubMed]
54. Rizk, F.H.; Gaber, K.S.; Eid, M.M.; Khafaga, D.S.; Alhussan, A.A.; El-kenawy, E.S.M. Dynamic binary swordfish movement optimization algorithm for feature selection. *J. Big Data* **2026**, *13*, 8. [CrossRef]
55. Xie, Y.; Li, W.; Qin, B.; Gao, S. An Enhanced Plant Growth Algorithm with Adam Learning, Lévy Flight, and Dynamic Stage Control. *Symmetry* **2025**, *18*, 64. [CrossRef]
56. Faramarzi, A.; Heidarinejad, M.; Mirjalili, S.; Gandomi, A.H. Marine Predators Algorithm: A nature-inspired metaheuristic. *Expert Syst. Appl.* **2020**, *152*, 113377. [CrossRef]
57. Tubishat, M.; Ja'afar, S.; Alswaitti, M.; Mirjalili, S.; Idris, N.; Ismail, M.A.; Omar, M.S. Dynamic salp swarm algorithm for feature selection. *Expert Syst. Appl.* **2021**, *164*, 113873. [CrossRef]
58. Zhou, J.; Wu, Q.; Zhou, M.; Wen, J.; Al-Turki, Y.; Abusorrah, A. LAGAM: A length-adaptive genetic algorithm with Markov blanket for high-dimensional feature selection in classification. *IEEE Trans. Cybern.* **2022**, *53*, 6858–6869. [CrossRef] [PubMed]
59. Piotrowski, A.P. L-SHADE optimization algorithms with population-wide inertia. *Inf. Sci.* **2018**, *468*, 117–141. [CrossRef]
60. Zhang, J.; Sanderson, A.C. JADE: Adaptive differential evolution with optional external archive. *IEEE Trans. Evol. Comput.* **2009**, *13*, 945–958. [CrossRef]